```
// ===== FILE: src/main.ts =====
/**
 * 花花妙屋 - 游戏入口
 */
// unsafe-eval patch 必须最先导入，在 new PIXI.Application() 之前执行
import '@/core/pixiUnsafeEvalPatch';
import { Game } from '@/core/Game';
import { SceneManager } from '@/core/SceneManager';
import { BoardManager } from '@/managers/BoardManager';
import { MergeCompanionManager } from '@/managers/MergeCompanionManager';
import { BuildingManager } from '@/managers/BuildingManager';
import { SaveManager } from '@/managers/SaveManager';
import { MerchShopManager } from '@/managers/MerchShopManager';
import { IdleManager } from '@/managers/IdleManager';
import { RoomLayoutManager } from '@/managers/RoomLayoutManager';
import { LevelManager } from '@/managers/LevelManager';
import { CloudSyncManager } from '@/managers/CloudSyncManager';
import { UserIdentityManager } from '@/managers/UserIdentityManager';
import { PersistService, type CloudImportInfo } from '@/core/PersistService';
import { Platform } from '@/core/PlatformService';
import { EventBus } from '@/core/EventBus';
import { CdnAssetService } from '@/core/CdnAssetService';
import { TextureCache } from '@/utils/TextureCache';
import { LoadingScreenOverlay } from '@/gameobjects/ui/LoadingScreenOverlay';
import { MainScene } from '@/scenes/MainScene';
import { ShopScene } from '@/scenes/ShopScene';
import { computeBoardMetrics } from '@/config/Constants';
declare const GameGlobal: any;
// 全局错误捕获——确保真机上所有异常可见
if (typeof GameGlobal !== 'undefined') {
  GameGlobal.onError = (msg: string) => {
    console.error('[GlobalError]', msg);
  };
  GameGlobal.onUnhandledRejection = (ev: any) => {
    console.error('[UnhandledRejection]', ev?.reason || ev);
  };
}
const BUILD_TIME = '__BUILD_' + new Date().toISOString().slice(0, 19) + '__';
async function main(): Promise<void> {
  try {
    console.log('[main] 花花妙屋启动中... BUILD:', BUILD_TIME);
    // 注意：不再在启动时无条件清除存档!
    // 旧存档的兼容性由 SaveManager 的指纹校验负责处理
    // 环境诊断
    console.log('[main] typeof document:', typeof document,
      ', typeof window:', typeof window,
      ', typeof setTimeout:', typeof setTimeout);
    // 获取主屏 canvas
    const canvas = (typeof GameGlobal !== 'undefined' && GameGlobal.canvas)
      || (typeof window !== 'undefined' && (window as any).canvas)
```

```
  || null;
if (!canvas) {
  throw new Error('[main] 无法获取 canvas，请检查 pixi-adapter 是否正确加载');
}
console.log('[main] canvas 获取成功, width:', canvas.width, 'height:', canvas.height);
// 初始化游戏
Game.init(canvas);
console.log('[main] Game.init 完成');
// EventSystem 诊断
try {
  const renderer = (Game as any).app?.renderer;
  const events = renderer?.events;
  console.log('[main] EventSystem:', !!events,
    'domElement:', !!events?.domElement,
    'supportsPointerEvents:', events?.supportsPointerEvents,
    'supportsTouchEvents:', events?.supportsTouchEvents);
} catch (e) { console.warn('[main] EventSystem 诊断失败:', e); }
// topReserved = safeTop + TopBar(60) + gap(4) + ShopArea(250); 与 MainScene.SHOP_HEIGHT 一致（全景视...
const topReserved = Game.safeTop + 60 + 4 + MainScene.SHOP_HEIGHT;
computeBoardMetrics(Game.logicHeight, topReserved);
console.log(`[main] BoardMetrics 计算完成, logicHeight:${Game.logicHeight}, safeTop:${Game.safeTo...
Game.stage.sortableChildren = true;
const loadingOverlay = new LoadingScreenOverlay();
Game.stage.addChild(loadingOverlay);
let initialSaveLoaded = false;
let runtimeReadyForCloudReload = false;
let pendingCloudReloadInfo: CloudImportInfo | null = null;
const handleCloudSaveReload = (info: CloudImportInfo): void => {
  console.warn(
    `[main] 云端核心存档已覆盖本地，准备重载/重启 reason=${info.reason}, updatedAt=${info.updatedAt}`,
  );
  Platform.showToast('已恢复云端存档，正在刷新', 'none');
  // 小游戏里重启最稳：能避免各业务 Manager 已绑定的旧内存状态继续写回。
  if (Platform.restartMiniProgram()) return;
  if (SaveManager.reloadFromStorage(`cloud-import:${info.reason}`)) {
    EventBus.emit('cloud:saveReloaded', info);
  }
};
PersistService.subscribeCloudImport((info) => {
  if (!info.changedKeys.includes('huahua_save')) return;
  if (!initialSaveLoaded) {
    // 启动读档前发生的云端导入会被 SaveManager.load() 直接读到，不需要重启。
    return;
  }
  if (!runtimeReadyForCloudReload) {
    pendingCloudReloadInfo = info;
    return;
  }
  handleCloudSaveReload(info);
});
```

```
101 await TextureCache.preloadLoadingSplash();
102 loadingOverlay.applySplashTexture();
103 loadingOverlay.applyTitleTexture();
104 await CdnAssetService.fetchManifest();
105 // 启动只阻塞关键资源：主包 UI / 新手故事 / 默认头像 + items 分包。
106 // 其它 CDN 大资源进入游戏后后台预热或按需懒加载，避免新号首进因全量 CDN 下载卡住或空白。
107 await TextureCache.preloadCritical((loaded, total) => {
108   const ratio = total > 0 ? loaded / total : 1;
109   loadingOverlay.setProgress(0.05 + ratio * 0.45);
110 });
111 await TextureCache.loadItemsSubpackage((loaded, total) => {
112   const ratio = total > 0 ? loaded / total : 1;
113   loadingOverlay.setProgress(0.5 + ratio * 0.45);
114 });
115 loadingOverlay.setProgress(1);
116 // 先等 audio 分包就绪再进主场景，避免 InnerAudioContext 在文件未落地时解码报错
117 const _apiAudio: any = typeof wx !== 'undefined' ? wx : typeof tt !== 'undefined' ? tt : null;
118 await new Promise<void>((resolve) => {
119   if (CdnAssetService.isCdnPath('subpkg_audio/bgm_main.mp3')) {
120     console.log('[main] audio 已 CDN 化，跳过 audio 分包加载');
121     resolve();
122     return;
123   }
124   if (!_apiAudio?.loadSubpackage) {
125     resolve();
126     return;
127   }
128   _apiAudio.loadSubpackage({
129     name: 'audio',
130     success: () => {
131       console.log('[main] audio 分包加载成功');
132       resolve();
133     },
134     fail: (err: any) => {
135       console.warn('[main] audio 分包加载失败:', err);
136       resolve();
137     },
138   });
139 });
140 // 勿在此处移除 Loading：其后仍有棋盘初始化、云同步、MainScene 首帧构建等，
141 // 过早销毁会只剩 renderer 底色 0xFFF5EE，表现为「白屏一闪」。见下方 switchTo 之后。
142 // 初始化棋盘数据
143 BoardManager.init();
144 console.log('[main] BoardManager.init 完成');
145 MergeCompanionManager.init();
146 const startupSync = await CloudSyncManager.awaitStartupSync();
147 console.log(`[main] 云同步启动结果: ${startupSync.status}, reason=${startupSync.reason}`);
148 // 尝试加载存档（开发阶段已在启动时清除，此处应返回 false）
149 const loaded = SaveManager.load();
150 initialSaveLoaded = true;
```

```
151 if (!loaded) {
152   BuildingManager.reset();
153   MerchShopManager.init();
154   MerchShopManager.bootstrapFresh();
155 }
156 console.log('[main]', loaded ? '存档加载成功' : '无存档，使用默认数据');
157 UserIdentityManager.init();
158 // 星级升档奖励须在首帧 addStar 前就绪（不依赖 MainScene 是否已构建）
159 LevelManager.init();
160 // 再次确认棋盘状态
161 const cells = BoardManager.cells;
162 const hasItems = cells.some(c => c.itemId !== null);
163 const hasFog = cells.some(c => c.state !== 'open');
164 console.log(`[main] 棋盘确认: cells=${cells.length}, hasItems=${hasItems}, hasFog=${hasFog}`);
165 // 注册场景
166 const mainScene = new MainScene();
167 const shopScene = new ShopScene();
168 SceneManager.register(mainScene);
169 SceneManager.register(shopScene);
170 console.log('[main] MainScene + ShopScene 已注册');
171 // 进入主场景（首帧 UI 已挂到 stage 后再撤掉 Loading，避免中间空窗期浅底色「白屏」）
172 SceneManager.switchTo('main');
173 Game.stage.removeChild(loadingOverlay);
174 loadingOverlay.destroy({ children: true });
175 void TextureCache.preloadSceneWarmup('shop');
176 setTimeout(() => {
177   void TextureCache.preloadPanelAssets('checkin');
178   void TextureCache.preloadPanelAssets('quest');
179   void TextureCache.preloadSceneWarmup('deco');
180   void TextureCache.preloadSceneWarmup('worldmap');
181 }, 1500);
182 setTimeout(() => {
183   void TextureCache.preloadPanelAssets('warehouse');
184   void TextureCache.preloadPanelAssets('mergeChain');
185   void TextureCache.preloadPanelAssets('collection');
186   void TextureCache.preloadPanelAssets('affinity');
187   void TextureCache.preloadPanelAssets('dressup');
188   void TextureCache.preloadPanelAssets('merchShop');
189   void TextureCache.preloadPanelAssets('flowerSignGacha');
190 }, 3500);
191 runtimeReadyForCloudReload = true;
192 if (pendingCloudReloadInfo) {
193   const info = pendingCloudReloadInfo;
194   pendingCloudReloadInfo = null;
195   handleCloudSaveReload(info);
196 }
197 // 监听小游戏生命周期：退到后台时保存状态
198 const _apiMain: any = typeof wx !== 'undefined' ? wx : typeof tt !== 'undefined' ? tt : null;
199 if (_apiMain) {
200   _apiMain.onHide?.(() => {
```

```
      console.log('[main] 游戏退到后台，保存状态');
      IdleManager.onHide();
      SaveManager.save();
      // 立即刷写装饰布局（防止防抖 timer 未触发导致位置丢失）
      RoomLayoutManager.saveNow();
      void CloudSyncManager.flushNow('app-hide');
    });
    _apiMain.onShow?.(() => {
      console.log('[main] 游戏回到前台');
      if (MerchShopManager.ensureUpToDate()) {
        SaveManager.save();
      }
    });
  }
  console.log('[main] 花花妙屋启动完成 BUILD:', BUILD_TIME);
 } catch (e: any) {
  const errMsg = (e && (e.message || e.errMsg)) || '';
  let raw = '';
  try { raw = JSON.stringify(e, Object.getOwnPropertyNames(e || {})); } catch (_) { raw = Strin...
  console.error(`[main] 启动失败: errMsg=${errMsg} raw=${raw}`, e);
 }
}
main();
// ===== FILE: src/core/PlatformService.ts =====
/**
 * 平台服务抽象层 - 统一封装微信/抖音双平台 API
 *
 * 所有平台特有调用（存储、广告、分享等）都通过本模块统一访问，
 * src/ 中不再需要各自 declare const wx / tt。
 */
declare const wx: any;
declare const tt: any;
declare const GameGlobal: any;
export type PlatformName = 'wechat' | 'douyin' | 'unknown';
class PlatformServiceClass {
 /** 当前平台名 */
 readonly name: PlatformName;
 /** 底层平台 API 对象（wx / tt / null） */
 private _api: any;
 constructor() {
  if (typeof wx !== 'undefined') {
    this._api = wx;
    this.name = 'wechat';
  } else if (typeof tt !== 'undefined') {
    this._api = tt;
    this.name = 'douyin';
  } else {
    this._api = null;
    this.name = 'unknown';
  }
```

```
  console.log(`[Platform] 当前平台: ${this.name}`);
}
/** 是否在小游戏环境中 */
get isMinigame(): boolean {
  return this._api !== null;
}
/** 是否微信平台 */
get isWechat(): boolean {
  return this.name === 'wechat';
}
/** 是否抖音平台 */
get isDouyin(): boolean {
  return this.name === 'douyin';
}
/**
 * 是否有可用的后端 HTTP 通道
 * - 微信 / 抖音小游戏：有原生 request API
 * - 浏览器：有全局 fetch
 */
get canUseBackend(): boolean {
  if (this._api && typeof this._api.request === 'function') return true;
  return typeof (globalThis as any).fetch === 'function';
}
/** 底层 API（慎用，优先使用封装方法） */
get api(): any {
  return this._api;
}
// 存储
getStorageSync(key: string): string | null {
  try {
    return this._api?.getStorageSync(key) || null;
  } catch (_) {
    return null;
  }
}
setStorageSync(key: string, value: string): void {
  try {
    this._api?.setStorageSync(key, value);
  } catch (_) {}
}
/** 异步写入本地存储（避免阻塞主线程） */
setStorageAsync(key: string, value: string): void {
  try {
    if (this._api?.setStorage) {
      this._api.setStorage({ key, data: value, fail() {} });
    } else {
      this._api?.setStorageSync(key, value);
    }
  } catch (_) {}
}
```

```
301 removeStorageSync(key: string): void {
302   try {
303     this._api?.removeStorageSync(key);
304   } catch (_) {}
305 }
306 // 通用 HTTP（对接 CloudBase HTTP 访问服务）
307 /**
308  * 跨平台 HTTP 请求，统一返回 { statusCode, data }。
309  * - 非 2xx 不会 reject（让上层自行判定业务错误码）
310  * - 网络失败 / 超时 / 解析异常才 reject
311  */
312 request(opts: {
313   url: string;
314   method?: 'GET' | 'POST' | 'PUT' | 'DELETE';
315   data?: unknown;
316   headers?: Record<string, string>;
317   timeoutMs?: number;
318 }): Promise<{ statusCode: number; data: any }> {
319   const method = (opts.method || 'POST').toUpperCase();
320   const headers: Record<string, string> = {
321     'content-type': 'application/json',
322     ...(opts.headers || {}),
323   };
324   const timeoutMs = opts.timeoutMs && opts.timeoutMs > 0 ? opts.timeoutMs : 10000;
325   // 微信开发者工具的 wx.request 底层会走一层内部 XMLHttpRequest 桥接。
326   // 在部分基础库版本里，即使请求成功，也会额外打印 "An object could not be cloned"。
327   // 工具环境优先使用 fetch，可绕过这层 DevTools 内部 XHR；真机仍使用 wx.request。
328   if (this._isDevtools() && typeof (globalThis as any).fetch === 'function') {
329     console.log(`[Platform.request] devtools fetch transport ${method} ${opts.url}`);
330     return this._requestViaFetch(opts.url, method, opts.data, headers, timeoutMs);
331   }
332   if (this._api && typeof this._api.request === 'function') {
333     return this._requestViaMiniApi(opts.url, method, opts.data, headers, timeoutMs);
334   }
335   if (typeof (globalThis as any).fetch === 'function') {
336     return this._requestViaFetch(opts.url, method, opts.data, headers, timeoutMs);
337   }
338   return Promise.reject(new Error('no http transport available'));
339 }
340 private _requestViaMiniApi(
341   url: string,
342   method: string,
343   data: unknown,
344   headers: Record<string, string>,
345   timeoutMs: number,
346 ): Promise<{ statusCode: number; data: any }> {
347   return new Promise((resolve, reject) => {
348     let done = false;
349     const startedAt = Date.now();
350     const requestId = Math.random().toString(36).slice(2, 8);
```

```
351     const requestData = data === undefined || typeof data === 'string'
352       ? data
353       : JSON.stringify(data);
354     const timer = setTimeout(() => {
355       if (done) return;
356       done = true;
357       reject(new Error(`request timeout after ${timeoutMs}ms: ${url}`));
358     }, timeoutMs);
359     try {
360       console.log(`[Platform.request#${requestId}] ${method} ${url}, data=${requestData ? Strin...
361       this._api.request({
362         url,
363         method,
364         data: requestData,
365         header: headers,
366         timeout: timeoutMs,
367         success: (res: any) => {
368           if (done) return;
369           done = true;
370           clearTimeout(timer);
371           console.log(
372             `[Platform.request#${requestId}] success ${method} ${url}, status=${res?.statusCode...
373           );
374           resolve({
375             statusCode: res?.statusCode ?? 0,
376             data: res?.data,
377           });
378         },
379         fail: (err: any) => {
380           if (done) return;
381           clearTimeout(timer);
382           let raw = '';
383           try { raw = JSON.stringify(err); } catch (_) { raw = String(err); }
384           const msg = err?.errMsg || err?.message || raw || String(err);
385           console.warn(
386             `[Platform.request#${requestId}] fail ${method} ${url}: ${msg}, cost=${Date.now() -...
387           );
388           const fetchFn = (globalThis as any).fetch as typeof fetch | undefined;
389           const canFallback =
390             typeof fetchFn === 'function'
391             && /request:fail/i.test(msg);
392           if (canFallback) {
393             console.warn('[Platform.request] wx.request 失败，尝试 fetch 兜底:', url);
394             void this._requestViaFetch(url, method, data, headers, timeoutMs)
395               .then((result) => {
396                 if (done) return;
397                 done = true;
398                 resolve(result);
399               })
400               .catch((e2) => {
```

```
401                if (done) return;
402                done = true;
403                const fb = e2 instanceof Error ? e2.message : String(e2);
404                reject(new Error(`request failed: ${msg}; url=${url}; raw=${raw}; fetchFallback...
405              });
406            return;
407          }
408          done = true;
409          reject(new Error(`request failed: ${msg}; url=${url}; raw=${raw}`));
410        },
411      });
412    } catch (e) {
413      if (done) return;
414      done = true;
415      clearTimeout(timer);
416      reject(e instanceof Error ? e : new Error(String(e)));
417    }
418  });
419 }
420 private _requestViaFetch(
421   url: string,
422   method: string,
423   data: unknown,
424   headers: Record<string, string>,
425   timeoutMs: number,
426 ): Promise<{ statusCode: number; data: any }> {
427   const fetchFn = (globalThis as any).fetch as typeof fetch;
428   const ctrl = typeof AbortController !== 'undefined' ? new AbortController() : null;
429   const timer = ctrl ? setTimeout(() => ctrl.abort(), timeoutMs) : null;
430   const init: any = {
431     method,
432     headers,
433     signal: ctrl ? ctrl.signal : undefined,
434   };
435   if (data !== undefined && method !== 'GET') {
436     init.body = typeof data === 'string' ? data : JSON.stringify(data);
437   }
438   console.log(`[Platform.fetch] ${method} ${url}, data=${init.body ? String(init.body).length :...
439   return fetchFn(url, init).then(async (res) => {
440     if (timer) clearTimeout(timer);
441     const text = await res.text();
442     let parsed: any = text;
443     if (text) {
444       try {
445         parsed = JSON.parse(text);
446       } catch (_) {
447         parsed = text;
448       }
449     }
450     console.log(`[Platform.fetch] success ${method} ${url}, status=${res.status}`);
```

```
    return { statusCode: res.status, data: parsed };
  }).catch((e) => {
    if (timer) clearTimeout(timer);
    console.warn(`[Platform.fetch] fail ${method} ${url}:`, e instanceof Error ? e.message : St...
    throw e instanceof Error ? e : new Error(String(e));
  });
}
private _isDevtools(): boolean {
  try {
    return this._api?.getSystemInfoSync?.()?.platform === 'devtools';
  } catch (_) {
    return false;
  }
}
private _getRequestEnvSummary(): string {
  try {
    const info = this._api?.getSystemInfoSync?.();
    if (!info) return `platform=${this.name}`;
    return [
      `platform=${this.name}`,
      `sys=${info.system || info.platform || 'unknown'}`,
      `sdk=${info.SDKVersion || 'unknown'}`,
      `brand=${info.brand || 'unknown'}`,
      `model=${info.model || 'unknown'}`,
    ].join(',');
  } catch (_) {
    return `platform=${this.name}`;
  }
}
/**
 * 平台登录换 code:
 *   微信 -> wx.login({ success: { code } })
 *   抖音 -> tt.login({ success: { code } })
 *   其他 -> 返回 ''（上层走匿名路径）
 */
loginCode(): Promise<string> {
  return new Promise((resolve) => {
    if (!this._api || typeof this._api.login !== 'function') {
      resolve('');
      return;
    }
    try {
      this._api.login({
        success: (res: any) => resolve(res?.code || ''),
        fail: () => resolve(''),
      });
    } catch (_) {
      resolve('');
    }
  });
```

```
501 }
502 // 系统信息
503 getSystemInfoSync(): any {
504   try {
505     return this._api?.getSystemInfoSync?.() || null;
506   } catch (_) {
507     return null;
508   }
509 }
510 getMenuButtonBoundingClientRect(): any {
511   try {
512     return this._api?.getMenuButtonBoundingClientRect?.() || null;
513   } catch (_) {
514     return null;
515   }
516 }
517 // 震动反馈（已全局关闭，保留 API 避免调用方报错）
518 /** @deprecated 产品要求关闭全部硬件震动，此方法为空操作 */
519 vibrateShort(_type?: 'light' | 'medium' | 'heavy'): void {}
520 /** @deprecated 产品要求关闭全部硬件震动，此方法为空操作 */
521 vibrateLong(): void {}
522 // 广告
523 /** 创建激励视频广告实例 */
524 createRewardedVideoAd(adUnitId: string): any {
525   try {
526     if (this.name === 'wechat') {
527       return this._api?.createRewardedVideoAd?.({ adUnitId });
528     } else if (this.name === 'douyin') {
529       return this._api?.createRewardedVideoAd?.({ adUnitId });
530     }
531   } catch (e) {
532     console.warn('[Platform] 创建激励视频广告失败:', e);
533   }
534   return null;
535 }
536 /** 创建 Banner 广告实例 */
537 createBannerAd(adUnitId: string, style: any): any {
538   try {
539     if (this.name === 'wechat') {
540       return this._api?.createBannerAd?.({ adUnitId, style });
541     } else if (this.name === 'douyin') {
542       return this._api?.createBannerAd?.({ adUnitId, style });
543     }
544   } catch (e) {
545     console.warn('[Platform] 创建Banner广告失败:', e);
546   }
547   return null;
548 }
549 /** 创建插屏广告实例 */
550 createInterstitialAd(adUnitId: string): any {
```

```
551   if (!this._isValidAdUnitId(adUnitId)) {
552     console.warn('[Platform] 插屏广告位未配置，跳过创建');
553     return null;
554   }
555   try {
556     if (this.name === 'wechat') {
557       return this._api?.createInterstitialAd?.({ adUnitId });
558     } else if (this.name === 'douyin') {
559       return this._api?.createInterstitialAd?.({ adUnitId });
560     }
561   } catch (e) {
562     console.warn('[Platform] 创建插屏广告失败:', e);
563   }
564   return null;
565 }
566 private _isValidAdUnitId(adUnitId: string): boolean {
567   if (!adUnitId) return false;
568   if (/[xyz]{6,}/i.test(adUnitId)) return false;
569   if (this.name === 'wechat') return /^adunit-[0-9a-f]{16}$/i.test(adUnitId);
570   if (this.name === 'douyin') return !/[xyz]{6,}/i.test(adUnitId) && adUnitId.length >= 8;
571   return false;
572 }
573 /**
574  * 创建微信「游戏圈」原生按钮。
575  * 仅微信小游戏支持；调用方通常会把它做成透明热区，视觉仍由 Canvas UI 自行绘制。
576  */
577 createGameClubButton(opts: {
578   type?: 'text' | 'image';
579   text?: string;
580   icon?: 'green' | 'white' | 'dark' | 'light';
581   image?: string;
582   style: Record<string, any>;
583 }): any {
584   try {
585     if (this.name === 'wechat') {
586       return this._api?.createGameClubButton?.(opts);
587     }
588   } catch (e) {
589     console.warn('[Platform] 创建游戏圈按钮失败:', e);
590   }
591   return null;
592 }
593 // 分享
594 /** 显式开启右上角菜单分享入口（微信真机需要，否则菜单里可能显示当前页面不可分享） */
595 showShareMenu(opts?: { withShareTicket?: boolean; menus?: string[] }): void {
596   try {
597     this._api?.showShareMenu?.({
598       withShareTicket: opts?.withShareTicket ?? true,
599       menus: opts?.menus ?? ['shareAppMessage', 'shareTimeline'],
600     });
```

```
  } catch (_) {}
}
/** 主动分享（fire-and-forget） */
shareAppMessage(opts: { title: string; imageUrl?: string; query?: string }): void {
  try {
    this._api?.shareAppMessage?.(opts);
  } catch (_) {}
}
/** 将当前游戏 Canvas 的一段区域导出为临时图片路径，供动态分享图使用 */
canvasToTempFilePath(opts: {
  canvas: any;
  x?: number;
  y?: number;
  width?: number;
  height?: number;
  destWidth?: number;
  destHeight?: number;
  fileType?: 'jpg' | 'png';
  quality?: number;
}): Promise<string | null> {
  return new Promise(resolve => {
    if (!this._api?.canvasToTempFilePath || !opts.canvas) {
      console.warn('[Platform] canvasToTempFilePath unavailable');
      resolve(null);
      return;
    }
    try {
      this._api.canvasToTempFilePath({
        canvas: opts.canvas,
        x: opts.x,
        y: opts.y,
        width: opts.width,
        height: opts.height,
        destWidth: opts.destWidth,
        destHeight: opts.destHeight,
        fileType: opts.fileType ?? 'jpg',
        quality: opts.quality ?? 0.9,
        success: (res: { tempFilePath?: string }) => resolve(res.tempFilePath ?? null),
        fail: (err: unknown) => {
          console.warn('[Platform] canvasToTempFilePath failed', err);
          resolve(null);
        },
      });
    } catch (err) {
      console.warn('[Platform] canvasToTempFilePath exception', err);
      resolve(null);
    }
  });
}
/**
```

```
651  * 主动分享并等待结果（通过 onHide/onShow 时间差判断）。
652  * 微信已移除分享成功回调，此方法用时间差启发式判断：
653  * 离开 >2s 视为分享成功，否则视为取消。
654  *
655  * 微信开发者工具：`platform === 'devtools'` 时分享往往不触发与真机一致的前后台切换，
656  * 或回到前台间隔 <2s，导致恒为「取消」。工具内调用 `shareAppMessage` 成功后直接视为成功，便于测转发解锁格等流程
657  *
658  * @returns true = 可能已分享，false = 取消或未分享
659  */
660 shareAndWait(opts: { title: string; imageUrl?: string; query?: string }): Promise<boolean> {
661   return new Promise<boolean>((resolve) => {
662     if (!this._api) {
663       resolve(true);
664       return;
665     }
666     const sys = this.getSystemInfoSync();
667     const isDevtools = sys?.platform === 'devtools';
668     if (isDevtools) {
669       try {
670         this._api.shareAppMessage(opts);
671         console.log('[Platform] shareAndWait: devtools 环境，分享调用后视为成功（真机仍走 onHide/onShow 启发式）');
672         resolve(true);
673       } catch (_) {
674         resolve(false);
675       }
676       return;
677     }
678     let hideTime = 0;
679     const SHARE_MIN_MS = 2000;
680     const onHide = () => {
681       hideTime = Date.now();
682     };
683     const onShow = () => {
684       cleanup();
685       const elapsed = Date.now() - hideTime;
686       resolve(elapsed >= SHARE_MIN_MS);
687     };
688     const cleanup = () => {
689       try { this._api?.offHide?.(onHide); } catch (_) {}
690       try { this._api?.offShow?.(onShow); } catch (_) {}
691     };
692     this._api.onHide(onHide);
693     this._api.onShow(onShow);
694     try {
695       this._api.shareAppMessage(opts);
696     } catch (_) {
697       cleanup();
698       resolve(false);
699     }
700   });
```

```
701 }
702 /** 注册被动分享（右上角"分享"） */
703 onShareAppMessage(callback: () => { title: string; imageUrl?: string; query?: string }): void {
704   try {
705     this._api?.onShareAppMessage?.(callback);
706   } catch (_) {}
707 }
708 /** 分享到朋友圈（仅微信） */
709 onShareTimeline(callback: () => { title: string; imageUrl?: string; query?: string }): void {
710   try {
711     if (this.name === 'wechat') {
712       this._api?.onShareTimeline?.(callback);
713     }
714   } catch (_) {}
715 }
716 // 生命周期
717 onHide(callback: () => void): void {
718   try {
719     this._api?.onHide?.(callback);
720   } catch (_) {}
721 }
722 onShow(callback: (res?: any) => void): void {
723   try {
724     this._api?.onShow?.(callback);
725   } catch (_) {}
726 }
727 // 其他
728 /** 获取当前时间戳（服务器校时备用，默认本地） */
729 now(): number {
730   return Date.now();
731 }
732 /** 显示原生 Toast */
733 showToast(title: string, icon: 'success' | 'none' | 'error' = 'none'): void {
734   try {
735     this._api?.showToast?.({ title, icon, duration: 2000 });
736   } catch (_) {}
737 }
738 restartMiniProgram(): boolean {
739   try {
740     if (typeof this._api?.restartMiniProgram !== 'function') return false;
741     this._api.restartMiniProgram();
742     return true;
743   } catch (e) {
744     console.warn('[Platform] restartMiniProgram 失败:', e);
745     return false;
746   }
747 }
748 /**
749  * 写入剪贴板（异步 API，失败走 fail 回调，不会 throw）
750  * 微信需：公众平台 → 设置 → 基本设置 → 用户隐私保护指引 中声明剪贴板用途，
```

```
751  * 否则 fail（如 errno 1026）；仍可从开发者工具 Console 手动复制。
752  */
753 setClipboardData(data: string): void {
754   try {
755     const fn = this._api?.setClipboardData;
756     if (typeof fn !== 'function') return;
757     fn.call(this._api, {
758       data,
759       success() {},
760       fail(err: any) {
761         console.warn(
762           '[Platform] setClipboardData 失败（多因未配置剪贴板隐私）：',
763           err?.errMsg || err,
764         );
765       },
766     });
767   } catch (e) {
768     console.warn('[Platform] setClipboardData:', e);
769   }
770 }
771 }
772 // 全局单例
773 const _global: any = typeof GameGlobal !== 'undefined' ? GameGlobal
774   : typeof globalThis !== 'undefined' ? globalThis : {};
775 if (!_global.__platformService) {
776   _global.__platformService = new PlatformServiceClass();
777 }
778 export const Platform: PlatformServiceClass = _global.__platformService;
779 // ===== FILE: src/core/BackendService.ts =====
780 /**
781  * 统一后端访问层（CloudBase HTTP 访问服务）
782  *
783  * - 跨平台：复用 PlatformService.request()（微信 wx.request / 抖音 tt.request / H5 fetch）
784  * - 身份：微信/抖音走原生 code 登录，H5 等走本地匿名 ID 登录；userId 形如 `wx:{openid}` / `dy:{openid}` / `anon:{uuid}`
785  * - Token：服务端签发 JWT，客户端缓存在本地；过期或 401 自动重新 login
786  */
787 import {
788   BACKEND_ANON_ID_KEY,
789   BACKEND_BASE_URL,
790   BACKEND_LOGIN_PATH,
791   BACKEND_PULL_PATH,
792   BACKEND_PUSH_PATH,
793   BACKEND_REQUEST_TIMEOUT_MS,
794   BACKEND_TOKEN_KEY,
795 } from '@/config/CloudConfig';
796 import { Platform } from './PlatformService';
797 export interface BackendPullResult {
798   userId: string;
799   platform: string;
800   exists: boolean;
```

```
801   schemaVersion: number;
802   updatedAt: number;
803   payload: Record<string, string>;
804   payloadKeys: string[];
805   clientFingerprint?: string;
806 }
807 export interface BackendPushPayload {
808   schemaVersion: number;
809   updatedAt: number;
810   /** 本地变更所基于的云端版本，用于服务端拒绝旧缓存覆盖。 */
811   baseRemoteUpdatedAt: number;
812   clientFingerprint: string;
813   payload: Record<string, string>;
814 }
815 export interface BackendPushResult {
816   userId: string;
817   updatedAt: number;
818   savedAt: number;
819   mode: 'insert' | 'update';
820   sizeBytes: number;
821 }
822 interface StoredToken {
823   token: string;
824   userId: string;
825   platform: string;
826   expiresAt: number;
827 }
828 /** 业务错误（非网络异常），携带 HTTP 状态码与业务错误码 */
829 export class BackendError extends Error {
830   readonly status: number;
831   readonly code: string;
832   readonly data?: any;
833   constructor(status: number, code: string, message: string, data?: any) {
834     super(message);
835     this.name = 'BackendError';
836     this.status = status;
837     this.code = code;
838     this.data = data;
839   }
840 }
841 class BackendServiceClass {
842   private _stored: StoredToken | null = null;
843   private _loginInflight: Promise<StoredToken> | null = null;
844   get available(): boolean {
845     return Platform.canUseBackend;
846   }
847   get userId(): string {
848     return this._stored?.userId || '';
849   }
850   get platform(): string {
```

```
  return this._stored?.platform || '';
}
/** 拿到有效 token；过期 / 无缓存会自动登录 */
async ensureToken(): Promise<StoredToken> {
  if (this._stored && this._stored.expiresAt - Date.now() > 60_000) {
    return this._stored;
  }
  const cached = this._loadTokenFromStorage();
  if (cached && cached.expiresAt - Date.now() > 60_000) {
    this._stored = cached;
    return cached;
  }
  if (this._loginInflight) return this._loginInflight;
  this._loginInflight = this._login()
    .then((t) => {
      this._loginInflight = null;
      return t;
    })
    .catch((e) => {
      this._loginInflight = null;
      throw e;
    });
  return this._loginInflight;
}
async pullSave(): Promise<BackendPullResult> {
  return this._callWithAuth<BackendPullResult>(BACKEND_PULL_PATH, {});
}
async pushSave(snapshot: BackendPushPayload): Promise<BackendPushResult> {
  return this._callWithAuth<BackendPushResult>(BACKEND_PUSH_PATH, snapshot);
}
/** 清空本地 token（排障用），不影响本地存档 */
clearToken(): void {
  this._stored = null;
  try {
    Platform.removeStorageSync(BACKEND_TOKEN_KEY);
  } catch (_) {}
}
// 内部
private async _login(): Promise<StoredToken> {
  const body = await this._buildLoginBody();
  const { status, data } = await this._request(BACKEND_LOGIN_PATH, body, undefined);
  if (status !== 200 || !data || data.ok !== true || !data.data?.token) {
    const code = data?.code || 'LOGIN_FAIL';
    const msg = data?.error || `login failed (status=${status})`;
    throw new BackendError(status, code, msg, data?.data);
  }
  const stored: StoredToken = {
    token: String(data.data.token),
    userId: String(data.data.userId || ''),
    platform: String(data.data.platform || body.platform),
```

```
    expiresAt: Number(data.data.expiresAt || 0),
  };
  this._stored = stored;
  this._saveTokenToStorage(stored);
  console.log(`[Backend] login ok userId=${stored.userId}`);
  return stored;
}
private async _buildLoginBody(): Promise<{ platform: string; code?: string; anonId?: string }> {
  if (Platform.isWechat) {
    const code = await Platform.loginCode();
    if (!code) throw new BackendError(0, 'NO_WX_CODE', 'wx.login 未返回 code');
    return { platform: 'wx', code };
  }
  if (Platform.isDouyin) {
    const code = await Platform.loginCode();
    if (!code) throw new BackendError(0, 'NO_TT_CODE', 'tt.login 未返回 code');
    return { platform: 'dy', code };
  }
  // H5 / 其他 -> 匿名
  const anonId = this._getOrCreateAnonId();
  return { platform: 'anon', anonId };
}
private _getOrCreateAnonId(): string {
  let id = '';
  try {
    id = Platform.getStorageSync(BACKEND_ANON_ID_KEY) || '';
  } catch (_) {}
  if (id) return id;
  id = 'anon_' + Date.now().toString(36) + '_' + Math.random().toString(36).slice(2, 10);
  try {
    Platform.setStorageSync(BACKEND_ANON_ID_KEY, id);
  } catch (_) {}
  return id;
}
private async _callWithAuth<T>(path: string, body: any): Promise<T> {
  const token = await this.ensureToken();
  const { status, data } = await this._request(path, body, token.token);
  if (status === 401) {
    // token 过期或被吊销：清掉重试一次
    this.clearToken();
    const retryToken = await this.ensureToken();
    const res2 = await this._request(path, body, retryToken.token);
    return this._unwrap<T>(res2.status, res2.data);
  }
  return this._unwrap<T>(status, data);
}
private _unwrap<T>(status: number, data: any): T {
  if (status === 200 && data && data.ok === true) {
    return data.data as T;
  }
```

```
    const code = data?.code || `HTTP_${status}`;
    const msg = data?.error || `request failed status=${status}`;
    throw new BackendError(status, code, msg, data?.data);
  }
  private async _request(
    path: string,
    body: any,
    token: string | undefined,
  ): Promise<{ status: number; data: any }> {
    const url = BACKEND_BASE_URL + path;
    const headers: Record<string, string> = {
      'content-type': 'application/json',
    };
    if (token) headers['authorization'] = `Bearer ${token}`;
    const res = await Platform.request({
      url,
      method: 'POST',
      data: body || {},
      headers,
      timeoutMs: BACKEND_REQUEST_TIMEOUT_MS,
    });
    return { status: res.statusCode, data: res.data };
  }
  private _loadTokenFromStorage(): StoredToken | null {
    try {
      const raw = Platform.getStorageSync(BACKEND_TOKEN_KEY);
      if (!raw) return null;
      const parsed = JSON.parse(raw);
      if (!parsed || typeof parsed.token !== 'string' || !parsed.token) return null;
      return {
        token: parsed.token,
        userId: parsed.userId || '',
        platform: parsed.platform || '',
        expiresAt: Number(parsed.expiresAt || 0),
      };
    } catch (_) {
      return null;
    }
  }
  private _saveTokenToStorage(stored: StoredToken): void {
    try {
      Platform.setStorageSync(BACKEND_TOKEN_KEY, JSON.stringify(stored));
    } catch (_) {}
  }
}
export const BackendService = new BackendServiceClass();
// ===== FILE: src/managers/CloudSyncManager.ts =====
/**
 * 云同步管理器 —— HTTP 版
 *
```

```
1001 * 对接 CloudBase HTTP 访问服务（见 src/core/BackendService.ts）。
1002 * 逻辑保留：
1003 *   - 启动时拉云端 + 本地合并（updatedAt 比较决定上/下行）
1004 *   - 本地脏标记订阅 + 防抖
1005 *   - 连续失败指数退避 / 低频重试
1006 *
1007 * 每平台独立身份（wx:/dy:/anon: 前缀），不互通；闸门改用 Platform.canUseBackend。
1008 */
1009 import {
1010   CLOUD_SYNC_BASE_DELAY_MS,
1011   CLOUD_SYNC_DEBOUNCE_MS,
1012   CLOUD_SYNC_LOG_THRESHOLD,
1013   CLOUD_SYNC_MAX_BACKOFF_MS,
1014   CLOUD_SYNC_MAX_FAIL_COUNT,
1015   CLOUD_SYNC_RETRY_INTERVAL_MS,
1016   CLOUD_SYNC_STARTUP_TIMEOUT_MS,
1017 } from '@/config/CloudConfig';
1018 import { BackendError, BackendService } from '@/core/BackendService';
1019 import { PersistService } from '@/core/PersistService';
1020 import { Platform } from '@/core/PlatformService';
1021 export type CloudAuthorityState = 'disabled' | 'unknown' | 'confirmedRemote' | 'cacheOnly';
1022 export interface CloudStartupSyncResult {
1023   status: 'disabled' | 'confirmed' | 'remote-applied' | 'cache-only';
1024   reason: string;
1025 }
1026 class CloudSyncManagerClass {
1027   private _initPromise: Promise<void> | null = null;
1028   private _startupPromise: Promise<void> | null = null;
1029   private _cloudReady = false;
1030   private _initDone = false;
1031   private _syncTimer: ReturnType<typeof setTimeout> | null = null;
1032   private _retryTimer: ReturnType<typeof setInterval> | null = null;
1033   private _syncFailCount = 0;
1034   private _syncDisabled = false;
1035   private _syncing = false;
1036   private _syncPending = false;
1037   private _authorityState: CloudAuthorityState = this.enabled ? 'unknown' : 'disabled';
1038   private _lastStartupRemoteApplied = false;
1039   constructor() {
1040     PersistService.subscribe((changedKeys) => {
1041       if (changedKeys.length === 0) return;
1042       this.scheduleSync(`dirty:${changedKeys.length}`);
1043     });
1044   }
1045   get enabled(): boolean {
1046     return BackendService.available;
1047   }
1048   get ready(): boolean {
1049     return this._cloudReady;
1050   }
```

```
get authorityState(): CloudAuthorityState {
  return this._authorityState;
}
get cacheOnly(): boolean {
  return this._authorityState === 'cacheOnly';
}
get userId(): string {
  return BackendService.userId;
}
prewarm(): void {
  if (!this.enabled) {
    console.log('[CloudSync] prewarm 跳过: enabled=false, platform=', Platform.name);
    return;
  }
  if (!this._startupPromise) {
    console.log('[CloudSync] prewarm 开始初始化');
    this._startupPromise = this._initialize();
  }
}
async awaitStartupSync(timeoutMs = CLOUD_SYNC_STARTUP_TIMEOUT_MS): Promise<CloudStartupSyncResu...
  if (!this.enabled) {
    this._authorityState = 'disabled';
    return { status: 'disabled', reason: 'backend-disabled' };
  }
  this.prewarm();
  if (!this._startupPromise) return { status: 'disabled', reason: 'startup-missing' };
  let timer: ReturnType<typeof setTimeout> | null = null;
  const result = await Promise.race([
    this._startupPromise
      .then(() => 'done' as const)
      .catch(() => 'done' as const),
    new Promise<void>((resolve) => {
      timer = setTimeout(resolve, timeoutMs);
    }).then(() => 'timeout' as const),
  ]);
  if (timer) clearTimeout(timer);
  if (result === 'timeout') {
    this._enterCacheOnly('startup-timeout');
    return { status: 'cache-only', reason: 'startup-timeout' };
  }
  if (this._authorityState === 'cacheOnly') {
    return { status: 'cache-only', reason: 'startup-pull-failed' };
  }
  return {
    status: this._lastStartupRemoteApplied ? 'remote-applied' : 'confirmed',
    reason: this._lastStartupRemoteApplied ? 'remote-imported' : 'cloud-confirmed',
  };
}
scheduleSync(reason = 'debounce'): void {
  if (!this.enabled) return;
```

```
1101    this.prewarm();
1102    if (this._authorityState === 'cacheOnly') {
1103      this._syncPending = true;
1104      console.warn(`[CloudSync] cacheOnly 禁止上行，已暂存同步请求 reason=${reason}`);
1105      return;
1106    }
1107    if (!this._initDone) {
1108      this._syncPending = true;
1109      return;
1110    }
1111    if (!this._cloudReady || this._syncDisabled) return;
1112    if (this._syncTimer) {
1113      clearTimeout(this._syncTimer);
1114    }
1115    const delay = this._syncFailCount > 0
1116      ? Math.min(CLOUD_SYNC_BASE_DELAY_MS * Math.pow(2, this._syncFailCount - 1), CLOUD_SYNC_MAX_...
1117      : CLOUD_SYNC_DEBOUNCE_MS;
1118    this._syncTimer = setTimeout(() => {
1119      this._syncTimer = null;
1120      void this._syncToCloud(reason);
1121    }, delay);
1122  }
1123  async flushNow(reason = 'manual'): Promise<void> {
1124    if (!this.enabled) return;
1125    this.prewarm();
1126    if (this._startupPromise) {
1127      try {
1128        await this._startupPromise;
1129      } catch (_) {}
1130    }
1131    if (this._authorityState === 'cacheOnly') {
1132      console.warn(`[CloudSync] cacheOnly 跳过立即上行 reason=${reason}`);
1133      return;
1134    }
1135    if (!this._cloudReady) return;
1136    if (this._syncTimer) {
1137      clearTimeout(this._syncTimer);
1138      this._syncTimer = null;
1139    }
1140    await this._syncToCloud(reason, true);
1141  }
1142  private async _initialize(): Promise<void> {
1143    if (this._initPromise) return this._initPromise;
1144    this._initPromise = (async () => {
1145      try {
1146        console.log('[CloudSync] 初始化中... platform:', Platform.name);
1147        await BackendService.ensureToken();
1148        this._cloudReady = !!BackendService.userId;
1149        if (!this._cloudReady) {
1150          console.warn('[CloudSync] 未能获取 token/userId，继续使用本地存档');
```

```
1151        this._enterCacheOnly('no-user-id');
1152        return;
1153      }
1154      console.log('[CloudSync] 云同步就绪! userId:', BackendService.userId);
1155      await this._pullFromCloudOnStartup();
1156    } catch (e) {
1157      console.warn('[CloudSync] 初始化失败，继续使用本地存档:', e);
1158      this._enterCacheOnly('init-failed');
1159    } finally {
1160      this._initDone = true;
1161      if (this._syncPending && this._cloudReady) {
1162        this._syncPending = false;
1163        this.scheduleSync('pending-after-init');
1164      }
1165    }
1166  })();
1167  return this._initPromise;
1168 }
1169 private async _pullFromCloudOnStartup(): Promise<void> {
1170  let remote;
1171  try {
1172    remote = await BackendService.pullSave();
1173  } catch (e) {
1174    console.warn('[CloudSync] 启动拉取失败，保持本地存档:', e);
1175    this._enterCacheOnly('startup-pull-failed');
1176    return;
1177  }
1178  const localSnapshot = PersistService.exportCloudSnapshot();
1179  const localMeta = PersistService.getCloudSyncMeta();
1180  if (!remote.exists) {
1181    if (localSnapshot.payloadKeys.length > 0) {
1182      console.warn(
1183        `[CloudSync] 云端无存档，按云端权威清空本地缓存 keys=${localSnapshot.payloadKeys.length}`,
1184      );
1185    }
1186    PersistService.importCloudSnapshot({
1187      updatedAt: 0,
1188      payload: {},
1189      reason: this._authorityState === 'cacheOnly' ? 'startup-late' : 'startup',
1190    });
1191    this._confirmRemoteBaseline(0, 'startup-no-remote-doc');
1192    this._lastStartupRemoteApplied = localSnapshot.payloadKeys.length > 0;
1193    return;
1194  }
1195  const remoteUpdatedAt = Number(remote.updatedAt) || 0;
1196  const remotePayloadKeys = Array.isArray(remote.payloadKeys)
1197    ? remote.payloadKeys
1198    : Object.keys(remote.payload || {});
1199  if (remotePayloadKeys.length === 0) {
1200    if (localSnapshot.payloadKeys.length > 0) {
```

```
1201        console.warn(
1202          `[CloudSync] 云端为空存档，按云端权威清空本地缓存 keys=${localSnapshot.payloadKeys.length}`,
1203        );
1204      }
1205      PersistService.importCloudSnapshot({
1206        updatedAt: remoteUpdatedAt,
1207        payload: {},
1208        reason: this._authorityState === 'cacheOnly' ? 'startup-late' : 'startup',
1209      });
1210      this._confirmRemoteBaseline(remoteUpdatedAt, 'startup-empty-remote-doc');
1211      this._lastStartupRemoteApplied = localSnapshot.payloadKeys.length > 0;
1212      return;
1213    }
1214    const hasKnownRemoteBaseline = localMeta.remoteUpdatedAt > 0;
1215    const shouldApplyRemote = remotePayloadKeys.length > 0
1216      && (
1217        localSnapshot.payloadKeys.length === 0
1218        || !hasKnownRemoteBaseline
1219        || remoteUpdatedAt > localMeta.remoteUpdatedAt
1220      );
1221    if (shouldApplyRemote) {
1222      PersistService.importCloudSnapshot({
1223        updatedAt: remoteUpdatedAt,
1224        payload: remote.payload || {},
1225        reason: this._authorityState === 'cacheOnly' ? 'startup-late' : 'startup',
1226      });
1227      this._confirmRemoteBaseline(remoteUpdatedAt, 'startup-remote-imported');
1228      this._lastStartupRemoteApplied = true;
1229      console.log(`[CloudSync] 启动期已从云端恢复 ${remotePayloadKeys.length} 个 key`);
1230      return;
1231    }
1232    this._confirmRemoteBaseline(remoteUpdatedAt, 'startup-remote-confirmed');
1233    if (PersistService.isCloudDirty()) {
1234      this.scheduleSync('startup-local-dirty');
1235    }
1236  }
1237  private async _syncToCloud(reason: string, force = false): Promise<void> {
1238    if (!this._cloudReady) return;
1239    if (!force && this._syncDisabled) return;
1240    if (this._authorityState === 'cacheOnly') {
1241      this._syncPending = true;
1242      console.warn(`[CloudSync] cacheOnly 拦截上行 reason=${reason}`);
1243      return;
1244    }
1245    if (this._syncing) {
1246      this._syncPending = true;
1247      return;
1248    }
1249    this._syncing = true;
1250    try {
```

```
1251    const snapshot = PersistService.exportCloudSnapshot();
1252    const hasDirty = PersistService.isCloudDirty();
1253    if (!hasDirty) {
1254      return;
1255    }
1256    if (snapshot.payloadKeys.length === 0) return;
1257    if (snapshot.updatedAt <= 0) {
1258      PersistService.touchCloudMeta();
1259    }
1260    const finalSnapshot = PersistService.exportCloudSnapshot();
1261    try {
1262      const res = await BackendService.pushSave({
1263        schemaVersion: finalSnapshot.schemaVersion,
1264        updatedAt: finalSnapshot.updatedAt,
1265        baseRemoteUpdatedAt: finalSnapshot.baseRemoteUpdatedAt,
1266        clientFingerprint: this._buildClientFingerprint(),
1267        payload: finalSnapshot.payload,
1268      });
1269      PersistService.markCloudSynced(res.updatedAt || finalSnapshot.updatedAt);
1270      this._confirmRemoteBaseline(res.updatedAt || finalSnapshot.updatedAt, 'push-ok');
1271      if (this._syncFailCount > 0) {
1272        console.log('[CloudSync] 云同步恢复成功');
1273      }
1274      console.log(
1275        `[CloudSync] 云端已同步 reason=${reason}, keys=${finalSnapshot.payloadKeys.length}, size=${r...
1276      );
1277      this._syncFailCount = 0;
1278      this._syncDisabled = false;
1279      if (this._retryTimer) {
1280        clearInterval(this._retryTimer);
1281        this._retryTimer = null;
1282      }
1283    } catch (e) {
1284      if (e instanceof BackendError && e.code === 'STALE_UPDATE' && e.data?.remote) {
1285        // 服务端版本更新 → 下行覆盖，重置失败计数
1286        const remote = e.data.remote as {
1287          updatedAt?: number;
1288          payload?: Record<string, string>;
1289        };
1290        console.warn('[CloudSync] 服务端版本更新，改为下行覆盖本地');
1291        PersistService.importCloudSnapshot({
1292          updatedAt: Number(remote.updatedAt) || Date.now(),
1293          payload: remote.payload || {},
1294          reason: 'stale-update',
1295        });
1296        this._confirmRemoteBaseline(Number(remote.updatedAt) || Date.now(), 'stale-update');
1297        this._syncFailCount = 0;
1298        this._syncDisabled = false;
1299        return;
1300      }
```

```
      throw e;
    }
  } catch (e: any) {
    this._syncFailCount += 1;
    if (this._syncFailCount <= CLOUD_SYNC_LOG_THRESHOLD) {
      console.warn(
        `[CloudSync] 云同步失败(${this._syncFailCount}/${CLOUD_SYNC_MAX_FAIL_COUNT}):`,
        e?.message || e,
      );
    }
    if (this._syncFailCount >= CLOUD_SYNC_MAX_FAIL_COUNT) {
      this._syncDisabled = true;
      if (!this._retryTimer) {
        console.warn('[CloudSync] 连续失败，进入低频重试模式（本地存档仍正常可用）');
        this._retryTimer = setInterval(() => {
          if (!this._syncing && PersistService.isCloudDirty()) {
            void this._syncToCloud('retry-interval', true);
          }
        }, CLOUD_SYNC_RETRY_INTERVAL_MS);
      }
    } else if (PersistService.isCloudDirty()) {
      this.scheduleSync(`retry-after-fail:${reason}`);
    }
  } finally {
    this._syncing = false;
    if (this._syncPending && !this._syncDisabled) {
      this._syncPending = false;
      this.scheduleSync('pending-resume');
    }
  }
}
private _buildClientFingerprint(): string {
  const info = Platform.getSystemInfoSync() || {};
  return [
    Platform.name,
    info.brand,
    info.model,
    info.platform,
    info.version,
  ]
    .filter(Boolean)
    .join('|')
    .slice(0, 160);
}
private _enterCacheOnly(reason: string): void {
  if (this._authorityState === 'cacheOnly') return;
  this._authorityState = 'cacheOnly';
  console.warn(`[CloudSync] 进入 cacheOnly，本地仅作缓存，禁止上行 reason=${reason}`);
}
private _confirmRemoteBaseline(remoteUpdatedAt: number, reason: string): void {
```

```
    this._authorityState = 'confirmedRemote';
    console.log(`[CloudSync] 云端基准已确认 reason=${reason}, remoteUpdatedAt=${remoteUpdatedAt}`);
  }
}
export const CloudSyncManager = new CloudSyncManagerClass();
// ===== FILE: src/config/AdConfig.ts =====
import { BOARD_COLS } from '@/config/Constants';
export type ExternalUnlockMode = 'share' | 'ad';
export interface PlatformAdUnitConfig {
  rewardedVideo: string;
  rewardedVideoByScene?: Record<string, string>;
  interstitial: string;
  banner: string;
}
/** 广告位 ID 配置：拿到真实 ID 后只改这里。 */
export const AD_UNIT_CONFIG: Record<'wechat' | 'douyin', PlatformAdUnitConfig> = {
  wechat: {
    rewardedVideo: 'adunit-6ca54692fdb75515',
    rewardedVideoByScene: {
      stamina_recover: 'adunit-e6e6362dc0a129f8',
      board_cell_unlock: 'adunit-73ce0cce0986d5b3',
      warehouse_slot_unlock: 'adunit-73ce0cce0986d5b3',
      special_deco_unlock: 'adunit-73ce0cce0986d5b3',
      /** 花漾沙发 / 原木茶几等宣传款家具（与通用解锁位分开） */
      promo_furniture_unlock: 'adunit-8cc5e94aff5578eb',
      merch_daily_refresh: 'adunit-6ca54692fdb75515',
      flower_sign_daily_draw: 'adunit-6ca54692fdb75515',
      warehouse_organize: 'adunit-05c339f10f72ed43',
      reward_box_organize: 'adunit-05c339f10f72ed43',
      cd_speedup: 'adunit-cea25a4d7c9202ae',
      merge_bubble_unlock: 'adunit-4091dd36e39990e6',
      /** 签到完成后加餐激励视频：与默认 rewardedVideo / 每日福利类一致 */
      checkin_ad_bonus: 'adunit-6ca54692fdb75515',
    },
    interstitial: 'adunit-yyyyyyyyyyyyyyyyyy',
    banner: 'adunit-zzzzzzzzzzzzzzzzzz',
  },
  douyin: {
    rewardedVideo: 'xxxxxxxxxxxxxxxxxxxxxxxxxx',
    interstitial: 'yyyyyyyyyyyyyyyyyyyyyyyyyy',
    banner: 'zzzzzzzzzzzzzzzzzzzzzzzzzz',
  },
};
/** 棋盘钥匙格逐格配置：未列出的钥匙格默认走分享。 */
export const BOARD_KEY_UNLOCK_MODES: Record<number, ExternalUnlockMode> = {
  [0 * BOARD_COLS + 1]: 'share',
  [0 * BOARD_COLS + 5]: 'ad',
  [4 * BOARD_COLS + 6]: 'share',
  [6 * BOARD_COLS + 3]: 'ad',
  [8 * BOARD_COLS + 0]: 'share',
```

```
  [8 * BOARD_COLS + 6]: 'ad',
};
/** 仓库前几个锁格逐格配置：index 为 0 基槽位下标。 */
export const WAREHOUSE_SLOT_UNLOCK_MODES: Record<number, ExternalUnlockMode> = {
  4: 'share',
  5: 'share',
  6: 'ad',
  7: 'share',
  8: 'ad',
  9: 'ad',
};
/** 广告可解锁购买资格的家具白名单；只有这里列出的家具走广告条件。 */
export const AD_UNLOCK_DECO_IDS = new Set<string>(['promo_floral_sofa', 'promo_wood_tea_table']);
/** 使用 `promo_furniture_unlock` 广告位的家具 id（其余走 `special_deco_unlock`） */
export const PROMO_FURNITURE_AD_DECO_IDS = new Set<string>(['promo_floral_sofa', 'promo_wood_tea_...
// ===== FILE: src/config/AffinityCardConfig.ts =====
/**
 * 友谊卡 + 图鉴 + 赛季 — 配置数据
 *
 * 主线流程（V2 · Bond 已退场）：
 *  - 玩家给「赛季客人」交付订单 → AffinityCardManager.rollCardDrop 决定是否掉卡
 *  - 新卡 → 直接入图鉴；
 *  - 重复卡 → 立即按 DUPLICATE_REWARDS 派发花愿/钻石/体力（不再积"碎片"）
 *  - 单客人集齐 50% / 100% → CUSTOMER_MILESTONE_REWARDS 自动结算
 *  - 整赛季 100%（所有客人都集齐）→ SEASON_GRAND_REWARD
 *
 * 接入点：
 *  - AffinityManager.onCustomerDelivered → AffinityCardManager.rollCardDrop
 *  - AffinityCardManager 内部：里程碑触发自己结算并发事件
 *
 * 美术：
 *  - 卡面 PNG：minigame/subpkg_chars/images/affinity_cards/card_<typeId>_<idx>.png
 *  - 缺图时 fallback 到 customer_<typeId> 大头照 + 卡框
 */
// ==============================================================================
// 稀有度
// ==============================================================================
export type CardRarity = 'N' | 'R' | 'SR' | 'SSR';
export const CARD_RARITIES: readonly CardRarity[] = ['N', 'R', 'SR', 'SSR'] as const;
/** 抽到时的稀有度概率分布（百分比，总和 100） */
export const CARD_RARITY_DROP_WEIGHTS: Record<CardRarity, number> = {
  N: 70,
  R: 22,
  SR: 7,
  SSR: 1,
};
/** 稀有度展示色（卡框边色 / 光环） */
export const CARD_RARITY_COLOR: Record<CardRarity, number> = {
  N: 0xB0BEC5,
  R: 0x64B5F6,
```

```
  SR: 0xBA68C8,
  SSR: 0xFFC107,
};
/** 稀有度展示中文 */
export const CARD_RARITY_LABEL: Record<CardRarity, string> = {
  N: '日常',
  R: '侧写',
  SR: '故事',
  SSR: '高光',
};
// ==============================================================================
// 抽卡参数
// ==============================================================================
/**
 * 友谊卡系统玩家解锁等级（含）。
 * - 新手前 5 级专注「合成 + 出花 + 卖花」核心循环
 * - Lv6 起开放「图鉴掉卡」，作为普通订单上的低频惊喜
 */
export const CARD_SYSTEM_UNLOCK_LEVEL = 6;
/** 一次「是否掉卡」的基础概率（普通订单）— 调低后约 1/10，留出惊喜感 */
export const CARD_DROP_BASE_CHANCE = 0.10;
/**
 * 每日全局掉卡上限。
 * 达到当日上限后所有客人订单当天都不再掉卡，但保底计数仍在累积，
 * 保证"每天最多 N 张惊喜，不打扰核心节奏"。
 * 按本地日历日切日（与签到 / 每日糖一致：UTC 偏移 + GM dateOffset）。
 */
export const CARD_DROP_DAILY_LIMIT = 3;
/** 保底：累计 N 次抽卡未出 SR/SSR → 强制 SR */
export const PITY_TO_SR_THRESHOLD = 30;
/** 保底：累计 N 次抽卡未出 SSR → 强制 SSR */
export const PITY_TO_SSR_THRESHOLD = 100;
// ==============================================================================
// 重复卡奖励
// ==============================================================================
export interface CardReward {
  huayuan?: number;
  diamond?: number;
  stamina?: number;
  flowerSignTickets?: number;
}
/** 重复卡按稀有度直发的奖励（V2：替代旧"友谊点碎片"） */
export const DUPLICATE_REWARDS: Record<CardRarity, CardReward> = {
  N:  { huayuan: 20 },
  R:  { huayuan: 80 },
  SR: { huayuan: 300, stamina: 3 },
  SSR: { huayuan: 1500, diamond: 30, stamina: 10 },
};
// ==============================================================================
// 单客人图鉴里程碑（每客 12 张卡 → 6 张 / 12 张两档）
```

```
57124        data[offset] = v[0];
57125        data[offset+1] = v[1];
57126        data[offset+2] = v[2];
57127        data[offset+3] = v[3];
57128    `,mat2:`
57129        data[offset] = v[0];
57130        data[offset+1] = v[1];
57131        data[offset+4] = v[2];
57132        data[offset+5] = v[3];
57133    `,mat3:`
57134        data[offset] = v[0];
57135        data[offset+1] = v[1];
57136        data[offset+2] = v[2];
57137        data[offset + 4] = v[3];
57138        data[offset + 5] = v[4];
57139        data[offset + 6] = v[5];
57140        data[offset + 8] = v[6];
57141        data[offset + 9] = v[7];
57142        data[offset + 10] = v[8];
57143    `,mat4:`
57144        for(var i = 0; i < 16; i++)
57145        {
57146            data[offset + i] = v[i];
57147        }
57148    `},Xf={float:4,vec2:8,vec3:12,vec4:16,int:4,ivec2:8,ivec3:12,ivec4:16,uint:4,uvec2:8,uvec3:12...
57149    var v = null;
57150    var v2 = null;
57151    var cv = null;
57152    var t = 0;
57153    var gl = renderer.gl
57154    var index = 0;
57155    var data = buffer.data;
57156    `];for(let r=0;r<i.length;r++){const a=i[r],l=o.uniforms[a.data.name],h=a.data.name;let c=!1;...
57157            cv = ud.${h}.value;
57158            v = uv.${h};
57159            offset = ${a.offset/4};
57160            t = 0;
57161            for(var i=0; i < ${a.data.size*u}; i++)
57162            {
57163                for(var j = 0; j < ${_}; j++)
57164                {
57165                    data[offset++] = v[t++];
57166                }
57167                offset += ${p};
57168            }
57169            `)}else{const d=F1[a.data.type];n.push(`
57170            cv = ud.${h}.value;
57171            v = uv.${h};
57172            offset = ${a.offset/4};
57173            ${d};
```

```
57174                `)}}return n.push(`
57175        renderer.buffer.update(buffer);
57176      `),{size:s,syncFunc:new Function("ud","uv","renderer","syncData","buffer",n.join(`
57177 `))}}let G1=0;const da={textureCount:0,uboCount:0};class Vn{constructor(t){this.destroyed=!1,this...
57178 attribute vec2 aTextureCoord;
57179 uniform mat3 projectionMatrix;
57180 varying vec2 vTextureCoord;
57181 void main(void)
57182 {
57183     gl_Position = vec4((projectionMatrix * vec3(aVertexPosition, 1.0)).xy, 0.0, 1.0);
57184     vTextureCoord = aTextureCoord;
57185 }`,tC=`attribute vec2 aVertexPosition;
57186 uniform mat3 projectionMatrix;
57187 varying vec2 vTextureCoord;
57188 uniform vec4 inputSize;
57189 uniform vec4 outputFrame;
57190 vec4 filterVertexPosition( void )
57191 {
57192     vec2 position = aVertexPosition * max(outputFrame.zw, vec2(0.)) + outputFrame.xy;
57193     return vec4((projectionMatrix * vec3(position, 1.0)).xy, 0.0, 1.0);
57194 }
57195 vec2 filterTextureCoord( void )
57196 {
57197     return aVertexPosition * (outputFrame.zw * inputSize.zw);
57198 }
57199 void main(void)
57200 {
57201     gl_Position = filterVertexPosition();
57202     vTextureCoord = filterTextureCoord();
57203 }
57204 `;const eC=J1,Jf=tC;class tm{constructor(t){this.renderer=t}contextChange(t){let e;if(this.render...
57205 uniform sampler2D uSampler;
57206 uniform float uAlpha;
57207 void main(void)
57208 {
57209    gl_FragColor = texture2D(uSampler, vTextureCoord) * uAlpha;
57210 }
57211 `;class _C extends $e{constructor(t=1){super(eC,uC,{uAlpha:1}),this.alpha=t}get alpha(){return th...
57212 `);function mC(o){const t=pC[o],e=t.length;let i=fC,s="";const n="gl_FragColor += texture2D(uSamp...
57213 `}return i=i.replace("%blur%",s),i=i.replace("%size%",o.toString()),i}const gC=`
57214     attribute vec2 aVertexPosition;
57215     uniform mat3 projectionMatrix;
57216     uniform float strength;
57217     varying vec2 vBlurTexCoords[%size%];
57218     uniform vec4 inputSize;
57219     uniform vec4 outputFrame;
57220     vec4 filterVertexPosition( void )
57221     {
57222         vec2 position = aVertexPosition * max(outputFrame.zw, vec2(0.)) + outputFrame.xy;
57223         return vec4((projectionMatrix * vec3(position, 1.0)).xy, 0.0, 1.0);
```

```
57224    }
57225    vec2 filterTextureCoord( void )
57226    {
57227        return aVertexPosition * (outputFrame.zw * inputSize.zw);
57228    }
57229    void main(void)
57230    {
57231        gl_Position = filterVertexPosition();
57232        vec2 textureCoord = filterTextureCoord();
57233        %blur%
57234    }`;function yC(o,t){const e=Math.ceil(o/2);let i=gC,s="",n;t?n="vBlurTexCoords[%index%] =  te...
57235 `}return i=i.replace("%blur%",s),i=i.replace("%size%",o.toString()),i}class $c extends $e{constru...
57236 uniform sampler2D uSampler;
57237 uniform float m[20];
57238 uniform float uAlpha;
57239 void main(void)
57240 {
57241    vec4 c = texture2D(uSampler, vTextureCoord);
57242    if (uAlpha == 0.0) {
57243        gl_FragColor = c;
57244        return;
57245    }
57246    // Un-premultiply alpha before applying the color matrix. See issue #3539.
57247    if (c.a > 0.0) {
57248      c.rgb /= c.a;
57249    }
57250    vec4 result;
57251    result.r = (m[0] * c.r);
57252        result.r += (m[1] * c.g);
57253        result.r += (m[2] * c.b);
57254        result.r += (m[3] * c.a);
57255        result.r += m[4];
57256    result.g = (m[5] * c.r);
57257        result.g += (m[6] * c.g);
57258        result.g += (m[7] * c.b);
57259        result.g += (m[8] * c.a);
57260        result.g += m[9];
57261    result.b = (m[10] * c.r);
57262       result.b += (m[11] * c.g);
57263       result.b += (m[12] * c.b);
57264       result.b += (m[13] * c.a);
57265       result.b += m[14];
57266    result.a = (m[15] * c.r);
57267       result.a += (m[16] * c.g);
57268       result.a += (m[17] * c.b);
57269       result.a += (m[18] * c.a);
57270       result.a += m[19];
57271    vec3 rgb = mix(c.rgb, result.rgb, uAlpha);
57272    // Premultiply alpha again.
57273    rgb *= result.a;
```

```
57274     gl_FragColor = vec4(rgb, result.a);
57275 }
57276 `;class Xc extends $e{constructor(){const t={m:new Float32Array([1,0,0,0,0,0,1,0,0,0,0,0,1,0,0,0,...
57277 varying vec2 vTextureCoord;
57278 uniform vec2 scale;
57279 uniform mat2 rotation;
57280 uniform sampler2D uSampler;
57281 uniform sampler2D mapSampler;
57282 uniform highp vec4 inputSize;
57283 uniform vec4 inputClamp;
57284 void main(void)
57285 {
57286   vec4 map =  texture2D(mapSampler, vFilterCoord);
57287   map -= 0.5;
57288   map.xy = scale * inputSize.zw * (rotation * map.xy);
57289   gl_FragColor = texture2D(uSampler, clamp(vec2(vTextureCoord.x + map.x, vTextureCoord.y + map.y)...
57290 }
57291 `,CC=`attribute vec2 aVertexPosition;
57292 uniform mat3 projectionMatrix;
57293 uniform mat3 filterMatrix;
57294 varying vec2 vTextureCoord;
57295 varying vec2 vFilterCoord;
57296 uniform vec4 inputSize;
57297 uniform vec4 outputFrame;
57298 vec4 filterVertexPosition( void )
57299 {
57300     vec2 position = aVertexPosition * max(outputFrame.zw, vec2(0.)) + outputFrame.xy;
57301     return vec4((projectionMatrix * vec3(position, 1.0)).xy, 0.0, 1.0);
57302 }
57303 vec2 filterTextureCoord( void )
57304 {
57305     return aVertexPosition * (outputFrame.zw * inputSize.zw);
57306 }
57307 void main(void)
57308 {
57309     gl_Position = filterVertexPosition();
57310     vTextureCoord = filterTextureCoord();
57311     vFilterCoord = ( filterMatrix * vec3( vTextureCoord, 1.0)  ).xy;
57312 }
57313 `;class TC extends $e{constructor(t,e){const i=new ae;t.renderable=!1,super(CC,vC,{mapSampler:t._...
57314 varying vec2 v_rgbNE;
57315 varying vec2 v_rgbSW;
57316 varying vec2 v_rgbSE;
57317 varying vec2 v_rgbM;
57318 varying vec2 vFragCoord;
57319 uniform sampler2D uSampler;
57320 uniform highp vec4 inputSize;
57321 /**
57322  Basic FXAA implementation based on the code on geeks3d.com with the
57323  modification that the texture2DLod stuff was removed since it's
```

```
unsupported by WebGL.
--
From:
https://github.com/mitsuhiko/webgl-meincraft
Copyright (c) 2011 by Armin Ronacher.
Some rights reserved.
Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions are
met:
* Redistributions of source code must retain the above copyright
notice, this list of conditions and the following disclaimer.
* Redistributions in binary form must reproduce the above
copyright notice, this list of conditions and the following
disclaimer in the documentation and/or other materials provided
with the distribution.
* The names of the contributors may not be used to endorse or
promote products derived from this software without specific
prior written permission.
THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
"AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
(INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
*/
#ifndef FXAA_REDUCE_MIN
#define FXAA_REDUCE_MIN   (1.0/ 128.0)
#endif
#ifndef FXAA_REDUCE_MUL
#define FXAA_REDUCE_MUL   (1.0 / 8.0)
#endif
#ifndef FXAA_SPAN_MAX
#define FXAA_SPAN_MAX     8.0
#endif
//optimized version for mobile, where dependent
//texture reads can be a bottleneck
vec4 fxaa(sampler2D tex, vec2 fragCoord, vec2 inverseVP,
        vec2 v_rgbNW, vec2 v_rgbNE,
        vec2 v_rgbSW, vec2 v_rgbSE,
        vec2 v_rgbM) {
    vec4 color;
    vec3 rgbNW = texture2D(tex, v_rgbNW).xyz;
    vec3 rgbNE = texture2D(tex, v_rgbNE).xyz;
    vec3 rgbSW = texture2D(tex, v_rgbSW).xyz;
    vec3 rgbSE = texture2D(tex, v_rgbSE).xyz;
```

```
57374     vec4 texColor = texture2D(tex, v_rgbM);
57375     vec3 rgbM  = texColor.xyz;
57376     vec3 luma = vec3(0.299, 0.587, 0.114);
57377     float lumaNW = dot(rgbNW, luma);
57378     float lumaNE = dot(rgbNE, luma);
57379     float lumaSW = dot(rgbSW, luma);
57380     float lumaSE = dot(rgbSE, luma);
57381     float lumaM  = dot(rgbM,  luma);
57382     float lumaMin = min(lumaM, min(min(lumaNW, lumaNE), min(lumaSW, lumaSE)));
57383     float lumaMax = max(lumaM, max(max(lumaNW, lumaNE), max(lumaSW, lumaSE)));
57384     mediump vec2 dir;
57385     dir.x = -((lumaNW + lumaNE) - (lumaSW + lumaSE));
57386     dir.y =  ((lumaNW + lumaSW) - (lumaNE + lumaSE));
57387     float dirReduce = max((lumaNW + lumaNE + lumaSW + lumaSE) *
57388                     (0.25 * FXAA_REDUCE_MUL), FXAA_REDUCE_MIN);
57389     float rcpDirMin = 1.0 / (min(abs(dir.x), abs(dir.y)) + dirReduce);
57390     dir = min(vec2(FXAA_SPAN_MAX, FXAA_SPAN_MAX),
57391           max(vec2(-FXAA_SPAN_MAX, -FXAA_SPAN_MAX),
57392              dir * rcpDirMin)) * inverseVP;
57393     vec3 rgbA = 0.5 * (
57394                 texture2D(tex, fragCoord * inverseVP + dir * (1.0 / 3.0 - 0.5)).xyz +
57395                 texture2D(tex, fragCoord * inverseVP + dir * (2.0 / 3.0 - 0.5)).xyz);
57396     vec3 rgbB = rgbA * 0.5 + 0.25 * (
57397                         texture2D(tex, fragCoord * inverseVP + dir * -0.5).xyz +
57398                         texture2D(tex, fragCoord * inverseVP + dir * 0.5).xyz);
57399     float lumaB = dot(rgbB, luma);
57400     if ((lumaB < lumaMin) || (lumaB > lumaMax))
57401        color = vec4(rgbA, texColor.a);
57402     else
57403        color = vec4(rgbB, texColor.a);
57404     return color;
57405 }
57406 void main() {
57407       vec4 color;
57408       color = fxaa(uSampler, vFragCoord, inputSize.zw, v_rgbNW, v_rgbNE, v_rgbSW, v_rgbSE, v_rgbM);
57409       gl_FragColor = color;
57410 }
57411 `,xC=`
57412 attribute vec2 aVertexPosition;
57413 uniform mat3 projectionMatrix;
57414 varying vec2 v_rgbNW;
57415 varying vec2 v_rgbNE;
57416 varying vec2 v_rgbSW;
57417 varying vec2 v_rgbSE;
57418 varying vec2 v_rgbM;
57419 varying vec2 vFragCoord;
57420 uniform vec4 inputSize;
57421 uniform vec4 outputFrame;
57422 vec4 filterVertexPosition( void )
57423 {
```

```
57424     vec2 position = aVertexPosition * max(outputFrame.zw, vec2(0.)) + outputFrame.xy;
57425     return vec4((projectionMatrix * vec3(position, 1.0)).xy, 0.0, 1.0);
57426 }
57427 void texcoords(vec2 fragCoord, vec2 inverseVP,
57428             out vec2 v_rgbNW, out vec2 v_rgbNE,
57429             out vec2 v_rgbSW, out vec2 v_rgbSE,
57430             out vec2 v_rgbM) {
57431     v_rgbNW = (fragCoord + vec2(-1.0, -1.0)) * inverseVP;
57432     v_rgbNE = (fragCoord + vec2(1.0, -1.0)) * inverseVP;
57433     v_rgbSW = (fragCoord + vec2(-1.0, 1.0)) * inverseVP;
57434     v_rgbSE = (fragCoord + vec2(1.0, 1.0)) * inverseVP;
57435     v_rgbM = vec2(fragCoord * inverseVP);
57436 }
57437 void main(void) {
57438   gl_Position = filterVertexPosition();
57439   vFragCoord = aVertexPosition * outputFrame.zw;
57440   texcoords(vFragCoord, inputSize.zw, v_rgbNW, v_rgbNE, v_rgbSW, v_rgbSE, v_rgbM);
57441 }
57442 `;class EC extends $e{constructor(){super(xC,SC)}}var RC=`precision highp float;
57443 varying vec2 vTextureCoord;
57444 varying vec4 vColor;
57445 uniform float uNoise;
57446 uniform float uSeed;
57447 uniform sampler2D uSampler;
57448 float rand(vec2 co)
57449 {
57450     return fract(sin(dot(co.xy, vec2(12.9898, 78.233))) * 43758.5453);
57451 }
57452 void main()
57453 {
57454     vec4 color = texture2D(uSampler, vTextureCoord);
57455     float randomValue = rand(gl_FragCoord.xy * uSeed);
57456     float diff = (randomValue - 0.5) * uNoise;
57457     // Un-premultiply alpha before applying the color matrix. See issue #3539.
57458     if (color.a > 0.0) {
57459         color.rgb /= color.a;
57460     }
57461     color.r += diff;
57462     color.g += diff;
57463     color.b += diff;
57464     // Premultiply alpha again.
57465     color.rgb *= color.a;
57466     gl_FragColor = color;
57467 }
57468 `;class IC extends $e{constructor(t=.5,e=Math.random()){super(Jf,RC,{uNoise:0,uSeed:0}),this.nois...
57469 ${d}`)}});return await Promise.all(l),n?s[r[0].src]:s}async unload(t){const e=Ei(t,i=>({alias:[i]...
57470   "use strict";
57471   const WHITE_PNG = "
57472   async function checkImageBitmap() {
57473     try {
```

```
57474      if (typeof createImageBitmap != "function")
57475        return !1;
57476      const imageBlob = await (await fetch(WHITE_PNG)).blob(), imageBitmap = await createImageBit...
57477      return imageBitmap.width === 1 && imageBitmap.height === 1;
57478    } catch {
57479      return !1;
57480    }
57481  }
57482  checkImageBitmap().then((result) => {
57483    self.postMessage(result);
57484  });
57485 })();
57486 `;let Qn=null,jc=class{constructor(){Qn||(Qn=URL.createObjectURL(new Blob([aT],{type:"application...
57487  "use strict";
57488  async function loadImageBitmap(url) {
57489    const response = await fetch(url);
57490    if (!response.ok)
57491      throw new Error(\`[WorkerManager.loadImageBitmap] Failed to fetch \${url}: \${response.stat...
57492    const imageBlob = await response.blob();
57493    return await createImageBitmap(imageBlob);
57494  }
57495  self.onmessage = async (event) => {
57496    try {
57497      const imageBitmap = await loadImageBitmap(event.data.data[0]);
57498      self.postMessage({
57499        data: imageBitmap,
57500        uuid: event.data.uuid,
57501        id: event.data.id
57502      }, [imageBitmap]);
57503    } catch (e) {
57504      self.postMessage({
57505        error: e,
57506        uuid: event.data.uuid,
57507        id: event.data.id
57508      });
57509    }
57510  };
57511 })();
57512 `;let Zn=null;class ym{constructor(){Zn||(Zn=URL.createObjectURL(new Blob([lT],{type:"application...
57513 Please use Assets.add({ alias, src, data, format, loadParser }) instead.`),r.push({alias:t,src:e,...
57514 uniform vec4 uColor;
57515 uniform sampler2D uSampler;
57516 void main(void)
57517 {
57518    gl_FragColor = texture2D(uSampler, vTextureCoord) * uColor;
57519 }
57520 `,gS=`attribute vec2 aVertexPosition;
57521 attribute vec2 aTextureCoord;
57522 uniform mat3 projectionMatrix;
57523 uniform mat3 translationMatrix;
```

```
57524 uniform mat3 uTextureMatrix;
57525 varying vec2 vTextureCoord;
57526 void main(void)
57527 {
57528     gl_Position = vec4((projectionMatrix * translationMatrix * vec3(aVertexPosition, 1.0)).xy, 0....
57529     vTextureCoord = (uTextureMatrix * vec3(aTextureCoord, 1.0)).xy;
57530 }
57531 `;class Nm extends zi{constructor(t,e){const i={uSampler:t,alpha:1,uTextureMatrix:ae.IDENTITY,uCo...
57532 varying vec4 vColor;
57533 uniform sampler2D uSampler;
57534 void main(void){
57535     vec4 color = texture2D(uSampler, vTextureCoord) * vColor;
57536     gl_FragColor = color;
57537 }`,wS=`attribute vec2 aVertexPosition;
57538 attribute vec2 aTextureCoord;
57539 attribute vec4 aColor;
57540 attribute vec2 aPositionCoord;
57541 attribute float aRotation;
57542 uniform mat3 translationMatrix;
57543 uniform vec4 uColor;
57544 varying vec2 vTextureCoord;
57545 varying vec4 vColor;
57546 void main(void){
57547     float x = (aVertexPosition.x) * cos(aRotation) - (aVertexPosition.y) * sin(aRotation);
57548     float y = (aVertexPosition.x) * sin(aRotation) + (aVertexPosition.y) * cos(aRotation);
57549     vec2 v = vec2(x, y);
57550     v = v + aPositionCoord;
57551     gl_Position = vec4((translationMatrix * vec3(v, 1.0)).xy, 0.0, 1.0);
57552     vTextureCoord = aTextureCoord;
57553     vColor = aColor * uColor;
57554 }
57555 `;class Hm extends ra{constructor(t){super(t),this.shader=null,this.properties=null,this.tempMatr...
57556 `:t,t}static getFromCache(t,e,i,s){let n=i[t];return typeof n!="number"&&(n=At._measureText(t,e,s...
57557 #define SHADER_NAME Tiling-Sprite-100
57558 precision lowp float;
57559 in vec2 vTextureCoord;
57560 out vec4 fragmentColor;
57561 uniform sampler2D uSampler;
57562 uniform vec4 uColor;
57563 uniform mat3 uMapCoord;
57564 uniform vec4 uClampFrame;
57565 uniform vec2 uClampOffset;
57566 void main(void)
57567 {
57568     vec2 coord = vTextureCoord + ceil(uClampOffset - vTextureCoord);
57569     coord = (uMapCoord * vec3(coord, 1.0)).xy;
57570     vec2 unclamped = coord;
57571     coord = clamp(coord, uClampFrame.xy, uClampFrame.zw);
57572     vec4 texSample = texture(uSampler, coord, unclamped == coord ? 0.0f : -32.0f);// lod-bias ver...
57573     fragmentColor = texSample * uColor;
```

```
57574 }
57575 `,LS=`#version 300 es
57576 #define SHADER_NAME Tiling-Sprite-300
57577 precision lowp float;
57578 in vec2 aVertexPosition;
57579 in vec2 aTextureCoord;
57580 uniform mat3 projectionMatrix;
57581 uniform mat3 translationMatrix;
57582 uniform mat3 uTransform;
57583 out vec2 vTextureCoord;
57584 void main(void)
57585 {
57586     gl_Position = vec4((projectionMatrix * translationMatrix * vec3(aVertexPosition, 1.0)).xy, 0....
57587     vTextureCoord = (uTransform * vec3(aTextureCoord, 1.0)).xy;
57588 }
57589 `,DS=`#version 100
57590 #ifdef GL_EXT_shader_texture_lod
57591     #extension GL_EXT_shader_texture_lod : enable
57592 #endif
57593 #define SHADER_NAME Tiling-Sprite-100
57594 precision lowp float;
57595 varying vec2 vTextureCoord;
57596 uniform sampler2D uSampler;
57597 uniform vec4 uColor;
57598 uniform mat3 uMapCoord;
57599 uniform vec4 uClampFrame;
57600 uniform vec2 uClampOffset;
57601 void main(void)
57602 {
57603     vec2 coord = vTextureCoord + ceil(uClampOffset - vTextureCoord);
57604     coord = (uMapCoord * vec3(coord, 1.0)).xy;
57605     vec2 unclamped = coord;
57606     coord = clamp(coord, uClampFrame.xy, uClampFrame.zw);
57607     #ifdef GL_EXT_shader_texture_lod
57608         vec4 texSample = unclamped == coord
57609             ? texture2D(uSampler, coord)
57610             : texture2DLodEXT(uSampler, coord, 0);
57611     #else
57612         vec4 texSample = texture2D(uSampler, coord);
57613     #endif
57614     gl_FragColor = texSample * uColor;
57615 }
57616 `,zm=`#version 100
57617 #define SHADER_NAME Tiling-Sprite-100
57618 precision lowp float;
57619 attribute vec2 aVertexPosition;
57620 attribute vec2 aTextureCoord;
57621 uniform mat3 projectionMatrix;
57622 uniform mat3 translationMatrix;
57623 uniform mat3 uTransform;
```

```
varying vec2 vTextureCoord;
void main(void)
{
    gl_Position = vec4((projectionMatrix * translationMatrix * vec3(aVertexPosition, 1.0)).xy, 0....
    vTextureCoord = (uTransform * vec3(aTextureCoord, 1.0)).xy;
}
`,OS=`#version 100
#define SHADER_NAME Tiling-Sprite-Simple-100
precision lowp float;
varying vec2 vTextureCoord;
uniform sampler2D uSampler;
uniform vec4 uColor;
void main(void)
{
    vec4 texSample = texture2D(uSampler, vTextureCoord);
    gl_FragColor = texSample * uColor;
}
`;const Ba=new ae;class Vm extends ra{constructor(t){super(t),t.runners.contextChange.add(this),t...
varying vec2 vTextureCoord;\r
uniform sampler2D uSampler;\r
\r
// Tint\r
uniform vec4 uColor;\r
\r
// on 2D applications fwidth is screenScale / glyphAtlasScale * distanceFieldRange\r
uniform float uFWidth;\r
\r
void main(void) {\r
\r
  // To stack MSDF and SDF we need a non-pre-multiplied-alpha texture.\r
  vec4 texColor = texture2D(uSampler, vTextureCoord);\r
\r
  // MSDF\r
  float median = texColor.r + texColor.g + texColor.b -\r
                min(texColor.r, min(texColor.g, texColor.b)) -\r
                max(texColor.r, max(texColor.g, texColor.b));\r
  // SDF\r
  median = min(median, texColor.a);\r
\r
  float screenPxDistance = uFWidth * (median - 0.5);\r
  float alpha = clamp(screenPxDistance + 0.5, 0.0, 1.0);\r
  if (median < 0.01) {\r
    alpha = 0.0;\r
  } else if (median > 0.99) {\r
    alpha = 1.0;\r
  }\r
\r
  // Gamma correction for coverage-like alpha\r
  float luma = dot(uColor.rgb, vec3(0.299, 0.587, 0.114));\r
  float gamma = mix(1.0, 1.0 / 2.2, luma);\r
```

```
57674  float coverage = pow(uColor.a * alpha, gamma);  \r
57675 \r
57676  // NPM Textures, NPM outputs\r
57677  gl_FragColor = vec4(uColor.rgb, coverage);\r
57678 }\r
57679 `,$S=`// Mesh material default fragment\r
57680 attribute vec2 aVertexPosition;\r
57681 attribute vec2 aTextureCoord;\r
57682 \r
57683 uniform mat3 projectionMatrix;\r
57684 uniform mat3 translationMatrix;\r
57685 uniform mat3 uTextureMatrix;\r
57686 \r
57687 varying vec2 vTextureCoord;\r
57688 \r
57689 void main(void)\r
57690 {\r
57691    gl_Position = vec4((projectionMatrix * translationMatrix * vec3(aVertexPosition, 1.0)).xy, 0....
57692 \r
57693    vTextureCoord = (uTextureMatrix * vec3(aTextureCoord, 1.0)).xy;\r
57694 }\r
57695 `;const Zm=[],Jm=[],tg=[],XS=class Lb extends A{constructor(t,e={}){super();const{align:i,tint:s,...
57696 `)||" ",h=Qm(l),c=this._maxWidth*t.size/e,d=t.distanceFieldType=="none"?Zm:Jm;let u=null,_=0,p=0...
57697 `){r.push(_),a.push(-1),p=Math.max(p,_),++f,++y,s.x=0,s.y+=t.lineHeight,u=null,T=0;continue}const...
57698 `&&(/(?:\s)/.test(v)&&(_=g),r.push(_),p=Math.max(p,_),a.push(-1));const S=[];for(let D=0;D<=f;D++...
57699          @font-face {
57700             font-family: "${e.family}";
57701             src: url('${e.dataSrc}');
57702             font-weight: ${e.weight};
57703             font-style: ${e.style};
57704             font-display: ${e.display};
57705          }`,this._stylesheet)}get stylesheet(){return this._stylesheet}set stylesheet(t){this....
57706 `),{fontSize:14,fill:7035722,fontFamily:O,lineHeight:22,align:"center"});u.anchor.set(.5,0),u.pos...
57707 ${t}`);return}const e=Math.ceil(t.length/ml);console.log(`${o}（共 ${e} 段，按序号拼接）`);for(let i=0;i<e;...
57708 ${t.slice(i*ml,(i+1)*ml)}`)}const QI=[" 数据重置"," 货币调整"," 等级调整"," 棋盘操作"," 增加物品"," 装修系统"," 系统测试"," 新...
57709 （棋盘满时物品进入收纳箱）`,"支付","取消"))return}be.unlockBubbleWithDiamond(t)?(dt.show("已获得泡泡里的物品"),this._clea
57710 当前钻石：${Math.floor(j.state.diamond)}`,`扩容（${t}）`,"取消")&&(We.expand()||dt.show("钻石不足"))}_getLineCol...
57711 ${e[1]}`:e,u=new k(d,{fontSize:13,fill:5125166,fontFamily:O,fontWeight:"bold",lineHeight:16,align...
57712 小时候，这里是我最喜欢的地方。`},{textureKey:"tutorial_story_2",text:`"小花，奶奶老了，花店快开不下去了。
57713 你愿意回来看看吗？"`},{textureKey:"tutorial_story_3",text:`我决定回来。推开门的那一刻，
57714 熟悉的花香扑面而来——
57715 花店还在，只是需要有人重新照料。`},{textureKey:"tutorial_story_4",text:"从今天起，我要让这家花店重新热闹起来！"}];class r
57716 试试把同样的铲子拖上去，
57717 就可以解锁空间并合成新工具了！`):a?(_=`把两把相同的铲子拖到一起，
57718 合成更高级的工具吧！`,u="合成工具"):d?(u="继续合成",_=`上面还有一个相同的水壶！
57719 拖过去合成新工具吧~`):(u="继续合成",_=`把两个水壶合在一起，
57720 合成更高级的育苗盘吧！`),this._showBubble({title:u,body:_,spotlightTop:h.y,spotlightBottom:h.y+h.h});const p...
57721 继续合成，升级成育苗盘就能培育花朵了~`,buttonText:"继续",onButton:()=>{this._clearOverlay(),this._showGuideMergeTool(...
57722 快试试~`,spotlightTop:e.y,spotlightBottom:e.y+e.h});const i=(s,n,r)=>{r.startsWith("flower_")&&(C.of...
57723 等客人来买的时候就能用到~`,buttonText:"继续",onButton:()=>kt.advanceTo(Bt.CUSTOMER1_ARRIVE)}))};C.on("building:...
```

```
57724 棋盘上有匹配的花就能交付~`,buttonText:"知道了",onButton:()=>kt.advanceTo(Bt.GUIDE_DELIVER1)})):(Ke.spawnScripted...
57725 刚好是合成的那朵~`,buttonText:"知道了",onButton:()=>kt.advanceTo(Bt.GUIDE_DELIVER2)}))}_showGuideDeliver(t){...
57726 点击客人完成交付吧~`,spotlightTop:a,spotlightBottom:a+l})},n=()=>{Ke.customers.some(a=>a.allSatisfied)&&s(...
57727 我们赚到了花愿~`,buttonText:"花愿是什么？",onButton:()=>this._showHuayuanExplain()}):this._showBubble({title:"...
57728 奶奶看到一定很开心~
57729 去装修一下花店吧！`,buttonText:"继续",onButton:()=>kt.advanceTo(Bt.SHOP_INTRO_DIALOG)})}_showHuayuanExplain(...
57730 完成客人的订单就能赚取花愿，
57731 花愿可以用来装修花店哦~`,buttonText:"继续培育",spotlightTop:t.y,spotlightBottom:t.y+t.h,onButton:()=>kt.advanceT...
57732 凑齐两朵一样的花就能合成更高级的~`,spotlightTop:e.y,spotlightBottom:e.y+e.h});let i=this._countFlowersOnBoard("fl...
57733 把它们合成一朵更高级的花吧~`,buttonText:"继续",onButton:()=>kt.advanceTo(Bt.GUIDE_MERGE_FLOWER)}))};C.on("buildi..
57734 就能合成更高级的花朵！`,spotlightTop:a.y,spotlightBottom:a.y+a.h});const l=(h,c,d)=>{d.startsWith("flower_")...
57735 更高级的花可以满足更多客人~`,buttonText:"继续",onButton:()=>kt.advanceTo(Bt.CUSTOMER2_ARRIVE)}))};C.on("board:me...
57736 用赚到的花愿来装修花店吧！
57737 花愿越多，能买的家具越多哦~`,buttonText:"去看看",spotlightTop:t.y,spotlightBottom:t.y+t.h,onButton:()=>kt.advance...
57738 加油，让花花妙屋成为小镇最棒的花店！`,{fontSize:17,fill:6048317,fontFamily:O,wordWrap:!0,wordWrapWidth:t-e*2,lineHe...
57739 虽然有点旧，但稍微装修一下就会很漂亮~`,buttonText:"开始装修",onButton:()=>kt.advanceTo(Bt.GUIDE_BUY_FURNITURE)
57740 挑选一件喜欢的家具吧！`,spotlightTop:p.y,spotlightBottom:p.y+p.h})})();let e=null;const i=()=>{this._clearOv...
57741 用花愿买下这件家具~`,hitW:138,hitH:58})}_showDecoUnlockPlaceGuideAt(t){this._showDecoFloatingGuideAt(t,{zI...
57742 到花店里摆放这件家具~`,hitW:156,hitH:50,dimAlpha:.38})}_clearDecoPurchaseOverlay(){var t;this._stopFingerAn...
57743 然后点击「完成装修」吧~`,showAvatar:!0});const t=()=>{e(),kt.advanceTo(Bt.SHOP_COMPLETE_DIALOG)};C.on("furni...
57744 客人们一定会喜欢的~`,buttonText:"回去继续做花束",onButton:()=>kt.advanceTo(Bt.SWITCH_BACK_MERGE)})}_showSwitchB
57745 继续完成客人的订单吧~`,spotlightTop:n.y,spotlightBottom:n.y+n.h}),this._cleanup=()=>{}}_drawSpotlightMask(t...
57746 「${t.quote}」
57747 —— 来自「花花妙屋」，每朵花都有一段故事`}shareCard(t){Xt.shareAppMessage({title:`${t.name} —— ${t.quote}`,query:`ca...
57748 varying vec2 vTextureCoord;
57749 uniform sampler2D uSampler;
57750 uniform vec4 inputPixel;
57751 uniform vec4 inputClamp;
57752 uniform vec3 uOutlineRgb;
57753 uniform float uThicknessPx;
57754 void main(void) {
57755     vec2 uv = vTextureCoord;
57756     vec2 d = inputPixel.zw * uThicknessPx;
57757     vec4 col = texture2D(uSampler, clamp(uv, inputClamp.xy, inputClamp.zw));
57758     float a = col.a;
57759     float ax = d.x;
57760     float ay = d.y;
57761     float amax = a;
57762     amax = max(amax, texture2D(uSampler, clamp(uv + vec2(ax, 0.0), inputClamp.xy, inputClamp.zw))...
57763     amax = max(amax, texture2D(uSampler, clamp(uv - vec2(ax, 0.0), inputClamp.xy, inputClamp.zw))...
57764     amax = max(amax, texture2D(uSampler, clamp(uv + vec2(0.0, ay), inputClamp.xy, inputClamp.zw))...
57765     amax = max(amax, texture2D(uSampler, clamp(uv - vec2(0.0, ay), inputClamp.xy, inputClamp.zw))...
57766     amax = max(amax, texture2D(uSampler, clamp(uv + vec2(ax, ay), inputClamp.xy, inputClamp.zw)).a);
57767     amax = max(amax, texture2D(uSampler, clamp(uv + vec2(-ax, ay), inputClamp.xy, inputClamp.zw))...
57768     amax = max(amax, texture2D(uSampler, clamp(uv + vec2(ax, -ay), inputClamp.xy, inputClamp.zw))...
57769     amax = max(amax, texture2D(uSampler, clamp(uv + vec2(-ax, -ay), inputClamp.xy, inputClamp.zw)...
57770     float outer = smoothstep(0.03, 0.12, amax) * (1.0 - smoothstep(0.06, 0.38, a));
57771     if (outer > 0.35) {
57772         float oa = max(a, outer);
57773         gl_FragColor = vec4(uOutlineRgb * oa, oa);
```

```
57774     } else {
57775         gl_FragColor = col;
57776     }
57777 }
57778 `;function sb(){const o=new $e(void 0,SB,{uOutlineRgb:new Float32Array([1,.96,.18]),uThicknessPx:...
57779 去装修面板解锁吧~`,{fontSize:16,fill:Z.TEXT_LIGHT,fontFamily:O,align:"center"});d.anchor.set(.5,.5),d.pos...
57780 点「全部」可查看已放置项`,{fontSize:15,fill:Z.TEXT_LIGHT,fontFamily:O,align:"center"});d.anchor.set(.5,.5),d....
57781 去装修面板购买吧~`,{fontSize:16,fill:Z.TEXT_LIGHT,fontFamily:O,align:"center"});a.anchor.set(.5,.5),a.pos...
57782 // ===== FILE: minigame/game.js =====
57783 // ====== 最早期诊断 ======
57784 // 鸿蒙上"卡在转圈、控制台出不来"的情况，用 wx.showModal 弹窗显示诊断信息
57785 var _diagMsgs = [];
57786 var _diagStart = Date.now();
57787 function _diag(msg) {
57788   var ts = Date.now() - _diagStart;
57789   var line = '[' + ts + 'ms] ' + msg;
57790   _diagMsgs.push(line);
57791   try { console.log(line); } catch(_) {}
57792 }
57793 function _dumpRecentXhr(reason) {
57794   try {
57795     var g = typeof GameGlobal !== 'undefined' ? GameGlobal : (typeof globalThis !== 'undefined' ?...
57796     var logs = g && g.__xhrDebugLogs;
57797     if (!logs || !logs.length) {
57798       console.warn('[XHR-DUMP]', reason, 'no xhr logs');
57799       return;
57800     }
57801     var start = Math.max(0, logs.length - 30);
57802     console.warn('[XHR-DUMP]', reason, 'recent=' + (logs.length - start) + '/' + logs.length);
57803     for (var i = start; i < logs.length; i++) {
57804       console.warn(logs[i]);
57805     }
57806   } catch (e) {
57807     try { console.warn('[XHR-DUMP] failed:', e); } catch (_) {}
57808   }
57809 }
57810 // 弹窗显示诊断信息（控制台出不来时的最后手段）
57811 function _showDiag() {
57812   try {
57813     if (typeof wx !== 'undefined' && wx.showModal) {
57814       wx.showModal({
57815         title: '启动诊断',
57816         content: _diagMsgs.join('\n'),
57817         showCancel: false
57818       });
57819     }
57820   } catch(_) {}
57821 }
57822 _diag('game.js 开始执行');
57823 // 获取系统信息（用弹窗展示，不依赖控制台）
```

```
57824 try {
57825   if (typeof wx !== 'undefined') {
57826     var _si = wx.getSystemInfoSync();
57827     _diag('platform:' + _si.platform + ' system:' + _si.system);
57828     _diag('brand:' + _si.brand + ' model:' + _si.model);
57829     _diag('wxver:' + _si.version + ' SDKver:' + _si.SDKVersion);
57830   }
57831 } catch(e) {
57832   _diag('getSystemInfo失败:' + e);
57833 }
57834 // 全局错误捕获——鸿蒙等设备 adapter 阶段崩溃无日志，必须最先注册
57835 try {
57836   if (typeof GameGlobal !== 'undefined') {
57837     GameGlobal.onError = function(msg) {
57838       _diag('onError:' + msg);
57839       _dumpRecentXhr('onError');
57840       _showDiag();
57841     };
57842     GameGlobal.onUnhandledRejection = function(ev) {
57843       _diag('unhandledRej:' + (ev && ev.reason || ev));
57844       _dumpRecentXhr('onUnhandledRejection');
57845       _showDiag();
57846     };
57847   }
57848 } catch(_) {}
57849 // ====== 加载 adapter ======
57850 _diag('加载 pixi-adapter...');
57851 try {
57852   require('./pixi-adapter/index');
57853   _diag('pixi-adapter OK');
57854 } catch (e) {
57855   _diag('pixi-adapter 失败!!:' + e);
57856   _showDiag();
57857 }
57858 // ====== 鸿蒙 Intl polyfill ======
57859 // 鸿蒙版微信 V8 引擎不含 ICU，Intl 对象不存在
57860 // PixiJS graphemeSegmenter 中 `Intl==null?...` 会触发 ReferenceError
57861 // 注入一个最小 Intl stub，让 PixiJS 安全降级到 [...str] 分割
57862 if (typeof Intl === 'undefined') {
57863   _diag('Intl不存在,注入polyfill');
57864   var _g = typeof GameGlobal !== 'undefined' ? GameGlobal : (typeof globalThis !== 'undefined' ? ...
57865   _g.Intl = {};
57866 }
57867 // ====== 加载 game-bundle ======
57868 _diag('加载 game-bundle...');
57869 try {
57870   require('./game-bundle.js');
57871   _diag('game-bundle OK');
57872 } catch (e) {
57873   _diag('game-bundle 失败!!:' + e);
```

```
57874   _showDiag();
57875 }
57876 _diag('全部加载完成');
57877 // 5秒后如果还没有渲染，自动弹窗显示诊断（兜底）
57878 setTimeout(function() {
57879   if (typeof GameGlobal !== 'undefined' && !GameGlobal.__gameRendered) {
57880     _diag('5秒超时 - 游戏未渲染');
57881     _showDiag();
57882   }
57883 }, 5000);
57884 // ===== FILE: minigame/weapp-adapter.js =====
57885 // 已迁移到 pixi-adapter/ 模块化适配器，此文件仅为兼容保留
57886 // ===== FILE: cloudfunctions/huahua-api/index.js =====
57887 /**
57888  * 花花妙屋统一后端（CloudBase 云函数 + HTTP 访问服务）
57889  *
57890  * 路由（全部 POST JSON）：
57891  *   /login       登录：wx/dy code2session / anon，签发 JWT
57892  *   /save/pull   拉取当前用户存档
57893  *   /save/push   上传当前用户存档（Upsert，updatedAt 防回写）
57894  *   /health      健康检查（无鉴权）
57895  *
57896  * 客户端可用两种方式命中：
57897  *   1) HTTP 访问服务：POST {envDomain}/huahua-api/login 等
57898  *      event 形如 { path, httpMethod, headers, body, isBase64Encoded }
57899  *   2) SDK callFunction：data: { action: 'login'|'save/pull'|'save/push', body: {...} }
57900  *
57901  * 环境变量（CloudBase 控制台 → 云函数 → 环境变量）：
57902  *   HUAHUA_JWT_SECRET            必填；签发/校验 JWT
57903  *   HUAHUA_WX_APPID / HUAHUA_WX_SECRET   微信 code2session（带 HUAHUA_ 前缀是为了
57904  *     避开 CloudBase 对 `WX_APPID`/`WX_SECRET` 这两个裸 Key 的系统脱敏，否则值会被清空）
57905  *   HUAHUA_TT_APPID / HUAHUA_TT_SECRET   抖音 code2session（同理加前缀）
57906  *   HUAHUA_SAVE_MAX_BYTES         可选，默认 262144（256KB）
57907  *   HUAHUA_TOKEN_TTL_SEC         可选，默认 604800（7d）
57908  */
57909 const { handleLogin } = require('./lib/auth');
57910 const { handlePull, handlePush } = require('./lib/save');
57911 const { respond, parseEvent, preflight } = require('./lib/http');
57912 const ROUTES = {
57913   'POST /health': async () => ({ ok: true, ts: Date.now() }),
57914   'POST /login': handleLogin,
57915   'POST /save/pull': handlePull,
57916   'POST /save/push': handlePush,
57917 };
57918 exports.main = async (event, context) => {
57919   try {
57920     // HTTP 访问服务的 OPTIONS 预检
57921     if (event && event.httpMethod === 'OPTIONS') {
57922       return preflight();
57923     }
```

```
    const req = parseEvent(event);
    const key = `${req.method} ${req.path}`;
    const handler = ROUTES[key];
    if (!handler) {
      return respond(404, { ok: false, code: 'NOT_FOUND', error: `no route: ${key}` });
    }
    const result = await handler(req, context);
    // handler 允许直接返回 HTTP 响应对象（含 statusCode）
    if (result && typeof result === 'object' && 'statusCode' in result) {
      return result;
    }
    return respond(200, { ok: true, data: result });
  } catch (e) {
    const code = e && e.code ? e.code : 'INTERNAL';
    const status = e && e.status ? e.status : 500;
    const message = (e && e.message) || String(e);
    console.error('[huahua-api] error:', code, message, e && e.stack);
    const out = { ok: false, code, error: message };
    if (e && e.data !== undefined) out.data = e.data;
    return respond(status, out);
  }
};
// ===== FILE: cloudfunctions/huahua-api/lib/auth.js =====
/**
 * 登录 / Token 认证
 *
 * - wx: 调 api.weixin.qq.com/sns/jscode2session
 * - dy: 调 developer.toutiao.com/api/apps/v2/jscode2session
 * - tap: 当前占位，仅校验 taptapUserId 非空
 * - anon: 直接把客户端传的 anonId 作为 userId 后缀
 */
const jwt = require('jsonwebtoken');
const { httpError } = require('./http');
const {
  getGameKey,
  getJwtSecret: _readJwtSecret,
  getTtlSec,
} = require('./config');
const SUPPORTED_PLATFORMS = new Set(['wx', 'dy', 'tap', 'anon']);
function getJwtSecret() {
  const s = _readJwtSecret();
  if (!s) {
    const gk = getGameKey().toUpperCase().replace(/[^A-Z0-9]/g, '_');
    throw httpError(500, 'NO_JWT_SECRET', `${gk}_JWT_SECRET (或 HUAHUA_JWT_SECRET) 未配置`);
  }
  return s;
}
async function handleLogin(req) {
  const body = req.body || {};
  const platform = String(body.platform || '').toLowerCase();
```

```
57974  if (!SUPPORTED_PLATFORMS.has(platform)) {
57975    throw httpError(400, 'BAD_PLATFORM', `unsupported platform: ${platform}`);
57976  }
57977  let platformUid = '';
57978  if (platform === 'wx') {
57979    platformUid = await wxCode2Openid(body.code);
57980  } else if (platform === 'dy') {
57981    platformUid = await ttCode2Openid(body.code);
57982  } else if (platform === 'tap') {
57983    const id = String(body.taptapUserId || '').trim();
57984    if (!id) throw httpError(400, 'NO_TAP_ID', 'taptapUserId 缺失');
57985    platformUid = id;
57986  } else if (platform === 'anon') {
57987    const id = String(body.anonId || '').trim();
57988    if (!id) throw httpError(400, 'NO_ANON_ID', 'anonId 缺失');
57989    if (!/^[A-Za-z0-9_\-:.]{8,128}$/.test(id)) {
57990      throw httpError(400, 'BAD_ANON_ID', 'anonId 非法');
57991    }
57992    platformUid = id;
57993  }
57994  const userId = `${platform}:${platformUid}`;
57995  const ttlSec = getTtlSec();
57996  const secret = getJwtSecret();
57997  const gameKey = getGameKey();
57998  const now = Math.floor(Date.now() / 1000);
57999  // gk = gameKey，验证时若 token gk 与当前函数 GAME_KEY 不一致则拒绝
58000  const token = jwt.sign(
58001    { sub: userId, plt: platform, gk: gameKey, iat: now },
58002    secret,
58003    { expiresIn: ttlSec },
58004  );
58005  return {
58006    token,
58007    userId,
58008    platform,
58009    gameKey,
58010    expiresAt: (now + ttlSec) * 1000,
58011    ttlSec,
58012  };
58013 }
58014 /** 供 save 层复用：从 Authorization 头解析 userId；失败抛 401 */
58015 function requireUser(req) {
58016  const authHeader = (req.headers && req.headers.authorization) || '';
58017  const m = /^Bearer\s+(.+)$/i.exec(authHeader);
58018  if (!m) throw httpError(401, 'NO_TOKEN', '缺少 Authorization: Bearer <token>');
58019  const token = m[1].trim();
58020  let payload;
58021  try {
58022    payload = jwt.verify(token, getJwtSecret());
58023  } catch (e) {
```

```
    throw httpError(401, 'BAD_TOKEN', e && e.message ? e.message : 'token 无效');
  }
  const userId = payload && payload.sub;
  if (!userId || typeof userId !== 'string' || !userId.includes(':')) {
    throw httpError(401, 'BAD_TOKEN', 'token sub 非法');
  }
  // 若 token 带 gk 字段，必须与当前函数 GAME_KEY 一致，防止跨游戏拿 token 打本游戏函数
  const currentGk = getGameKey();
  if (payload.gk && payload.gk !== currentGk) {
    throw httpError(401, 'BAD_TOKEN', `token gameKey=${payload.gk} 与当前 GAME_KEY=${currentGk} 不匹配`);
  }
  return { userId, platform: payload.plt || userId.split(':')[0] };
}
async function wxCode2Openid(code) {
  // CloudBase/SCF 在某些环境会把 Key 为 `WX_APPID` / `WX_SECRET` 的值强制清空
  // （怀疑被当成小程序 AppID 保留字脱敏处理），所以这里优先读带游戏前缀的
  // `HUAHUA_WX_APPID` / `HUAHUA_WX_SECRET`，老的 `WX_APPID` / `WX_SECRET` 仅作兜底。
  const appid = process.env.HUAHUA_WX_APPID || process.env.WX_APPID;
  const secret = process.env.HUAHUA_WX_SECRET || process.env.WX_SECRET;
  if (!appid || !secret) throw httpError(500, 'NO_WX_CFG', 'HUAHUA_WX_APPID/HUAHUA_WX_SECRET 未配置');
  if (!code) throw httpError(400, 'NO_CODE', 'wx code 缺失');
  const url = `https://api.weixin.qq.com/sns/jscode2session?appid=${encodeURIComponent(appid)}&se...
  const data = await httpGetJson(url);
  if (!data || !data.openid) {
    throw httpError(401, 'WX_LOGIN_FAIL', `wx code2session 失败: ${JSON.stringify(data || {})}`);
  }
  return data.openid;
}
async function ttCode2Openid(code) {
  // 同 wxCode2Openid，抖音这组 env 也统一加游戏前缀，避免平台保留字冲突。
  const appid = process.env.HUAHUA_TT_APPID || process.env.TT_APPID;
  const secret = process.env.HUAHUA_TT_SECRET || process.env.TT_SECRET;
  if (!appid || !secret) throw httpError(500, 'NO_TT_CFG', 'HUAHUA_TT_APPID/HUAHUA_TT_SECRET 未配置');
  if (!code) throw httpError(400, 'NO_CODE', 'dy code 缺失');
  const url = 'https://developer.toutiao.com/api/apps/v2/jscode2session';
  const data = await httpPostJson(url, { appid, secret, code });
  // 抖音返回结构：{ err_no, err_tips, data: { openid, session_key, ... } }
  if (!data || data.err_no !== 0 || !data.data || !data.data.openid) {
    throw httpError(401, 'TT_LOGIN_FAIL', `dy code2session 失败: ${JSON.stringify(data || {})}`);
  }
  return data.data.openid;
}
function httpGetJson(url) {
  return httpRequestJson(url, 'GET');
}
function httpPostJson(url, body) {
  return httpRequestJson(url, 'POST', body);
}
function httpRequestJson(url, method, body) {
  // Node 18+ 内置 fetch；CloudBase 函数默认 Node 16/18+，若低版本将抛错提示升级
```

```
58074  if (typeof fetch !== 'function') {
58075    return Promise.reject(httpError(500, 'NO_FETCH', '当前 Node 运行时不支持 fetch，请使用 Node 18+'));
58076  }
58077  const opts = { method, headers: { 'Content-Type': 'application/json' } };
58078  if (body !== undefined) opts.body = JSON.stringify(body);
58079  return fetch(url, opts).then(async (res) => {
58080    const text = await res.text();
58081    try {
58082      return JSON.parse(text);
58083    } catch (_) {
58084      return { _raw: text };
58085    }
58086  });
58087 }
58088 module.exports = {
58089  handleLogin,
58090  requireUser,
58091 };
58092 // ===== FILE: cloudfunctions/huahua-api/lib/config.js =====
58093 /**
58094  * 游戏级配置（多游戏复用模板的唯一可变点）
58095  *
58096  * 其它游戏复用本后端模板时，**只改这里读取到的环境变量**:
58097  *   GAME_KEY            游戏代号，如 'huahua' / 'snake' / 'monaco'
58098  *                       决定：集合前缀 ${GAME_KEY}_xxx、JWT payload 里的 gk
58099  *   {GAMEKEY_UPPER}_JWT_SECRET   当前游戏的 JWT 签名密钥
58100  *                       例如 GAME_KEY=huahua → HUAHUA_JWT_SECRET
58101  *                       （向后兼容：找不到时回退 HUAHUA_JWT_SECRET）
58102  *   {GAMEKEY_UPPER}_TOKEN_TTL_SEC  token 有效期（秒）
58103  *                       （向后兼容：回退 HUAHUA_TOKEN_TTL_SEC，再回退 7d）
58104  *   {GAMEKEY_UPPER}_SAVE_MAX_BYTES payload 上限（字节）
58105  *                       （向后兼容：回退 HUAHUA_SAVE_MAX_BYTES，再回退 256KB）
58106  *   WX_APPID / WX_SECRET       微信 code2session 凭证
58107  *   TT_APPID / TT_SECRET       抖音 code2session 凭证
58108  *   TCB_ENV                    CloudBase 环境 ID（默认取 SYMBOL_CURRENT_ENV）
58109  *
58110  * 集合命名约定:
58111  *   ${GAME_KEY}_playerData  存档主表
58112  *   ${GAME_KEY}_xxx        未来扩展表（orders / rewards / inventory …）
58113  *
58114  * 云存储 key 前缀约定:
58115  *   ${GAME_KEY}/xxx.png    按游戏目录隔离
58116  */
58117 const DEFAULT_GAME_KEY = 'huahua';
58118 const DEFAULT_TTL_SEC = 7 * 24 * 3600;
58119 const DEFAULT_MAX_BYTES = 256 * 1024;
58120 function getGameKey() {
58121  const v = String(process.env.GAME_KEY || '').trim().toLowerCase();
58122  if (!v) return DEFAULT_GAME_KEY;
58123  if (!/^[a-z][a-z0-9_\-]{0,31}$/.test(v)) {
```

```
    throw new Error(`GAME_KEY 非法: "${v}" （要求小写字母开头，字母数字/下划线/连字符，长度 1~32）`);
  }
  return v;
}
function gameKeyUpper() {
  return getGameKey().toUpperCase().replace(/[^A-Z0-9]/g, '_');
}
function readEnvPrefer(...keys) {
  for (const k of keys) {
    const v = process.env[k];
    if (v !== undefined && v !== null && String(v).length > 0) return String(v);
  }
  return '';
}
function getCollectionName(suffix) {
  // 整体覆盖（向后兼容 / 临时切换）： HUAHUA_COLLECTION='xxx' 时直接用
  if (suffix === 'playerData' && process.env.HUAHUA_COLLECTION) {
    return String(process.env.HUAHUA_COLLECTION);
  }
  return `${getGameKey()}_${suffix}`;
}
function getJwtSecret() {
  const gk = gameKeyUpper();
  const v = readEnvPrefer(`${gk}_JWT_SECRET`, 'HUAHUA_JWT_SECRET');
  return v;
}
function getTtlSec() {
  const gk = gameKeyUpper();
  const raw = readEnvPrefer(`${gk}_TOKEN_TTL_SEC`, 'HUAHUA_TOKEN_TTL_SEC');
  const v = Number(raw);
  return Number.isFinite(v) && v > 0 ? Math.floor(v) : DEFAULT_TTL_SEC;
}
function getMaxBytes() {
  const gk = gameKeyUpper();
  const raw = readEnvPrefer(`${gk}_SAVE_MAX_BYTES`, 'HUAHUA_SAVE_MAX_BYTES');
  const v = Number(raw);
  return Number.isFinite(v) && v > 0 ? Math.floor(v) : DEFAULT_MAX_BYTES;
}
module.exports = {
  getGameKey,
  getCollectionName,
  getJwtSecret,
  getTtlSec,
  getMaxBytes,
};
// ===== FILE: cloudfunctions/huahua-api/lib/db.js =====
/**
 * CloudBase SDK 封装：复用同一个 app 与 collection 实例。
 * 集合名由 lib/config.js 的 GAME_KEY 派生，便于多游戏共用同一个 CloudBase 环境。
 */
```

```
58174 const tcb = require('@cloudbase/node-sdk');
58175 const { getCollectionName } = require('./config');
58176 let _app = null;
58177 function getApp() {
58178   if (_app) return _app;
58179   _app = tcb.init({
58180     env: process.env.TCB_ENV || tcb.SYMBOL_CURRENT_ENV,
58181   });
58182   return _app;
58183 }
58184 function getDb() {
58185   return getApp().database();
58186 }
58187 function getCollection() {
58188   return getDb().collection(getCollectionName('playerData'));
58189 }
58190 module.exports = {
58191   getApp,
58192   getDb,
58193   getCollection,
58194 };
58195 // ===== FILE: cloudfunctions/huahua-api/lib/http.js =====
58196 /**
58197  * HTTP 层工具：响应封装 / CORS / event 解析。
58198  * 同时兼容两种触发方式：
58199  *   A) HTTP 访问服务：event 含 httpMethod/path/body
58200  *   B) SDK callFunction：event 为 { action, body }，无 httpMethod
58201  */
58202 const CORS_HEADERS = {
58203   'Access-Control-Allow-Origin': '*',
58204   'Access-Control-Allow-Methods': 'GET,POST,OPTIONS',
58205   'Access-Control-Allow-Headers': 'Content-Type,Authorization',
58206   'Access-Control-Max-Age': '86400',
58207 };
58208 function respond(statusCode, body, extraHeaders = {}) {
58209   return {
58210     statusCode,
58211     headers: {
58212       'Content-Type': 'application/json; charset=utf-8',
58213       ...CORS_HEADERS,
58214       ...extraHeaders,
58215     },
58216     body: typeof body === 'string' ? body : JSON.stringify(body),
58217     isBase64Encoded: false,
58218   };
58219 }
58220 function preflight() {
58221   return {
58222     statusCode: 204,
58223     headers: { ...CORS_HEADERS },
```

```
    body: '',
    isBase64Encoded: false,
  };
}
function parseEvent(event) {
  event = event || {};
  // A) HTTP 访问服务
  if (event.httpMethod) {
    const method = String(event.httpMethod).toUpperCase();
    // CloudBase HTTP 访问服务默认把挂载路径前缀一起给 path，需要兼容性剥离
    // 例如挂载在 /huahua-api，客户端请求 /huahua-api/login，event.path 可能是 /huahua-api/login 或 /login
    let path = event.path || '/';
    path = normalizePath(path);
    let rawBody = event.body || '';
    if (event.isBase64Encoded && rawBody) {
      try {
        rawBody = Buffer.from(rawBody, 'base64').toString('utf8');
      } catch (_) {}
    }
    let body = {};
    if (rawBody) {
      try {
        body = JSON.parse(rawBody);
      } catch (_) {
        body = {};
      }
    }
    const headers = lowercaseHeaders(event.headers || {});
    return {
      method,
      path,
      body,
      headers,
      query: event.queryStringParameters || {},
      raw: event,
    };
  }
  // B) SDK callFunction 兼容入口：event = { action, body }
  const action = (event.action || '').replace(/^\/+/, '');
  const path = action ? `/${action}` : '/';
  return {
    method: 'POST',
    path,
    body: event.body || {},
    headers: lowercaseHeaders(event.headers || {}),
    query: {},
    raw: event,
  };
}
function normalizePath(path) {
```

```
  if (!path) return '/';
  let p = String(path);
  if (!p.startsWith('/')) p = '/' + p;
  // 剥离常见挂载前缀
  p = p.replace(/^\/(?:huahua-api)(?=\/|$)/, '');
  if (p === '') p = '/';
  // 去尾 /
  if (p.length > 1 && p.endsWith('/')) p = p.slice(0, -1);
  return p;
}
function lowercaseHeaders(h) {
  const out = {};
  for (const k of Object.keys(h || {})) {
    out[k.toLowerCase()] = h[k];
  }
  return out;
}
function httpError(status, code, message) {
  const err = new Error(message || code);
  err.status = status;
  err.code = code;
  return err;
}
module.exports = {
  respond,
  preflight,
  parseEvent,
  httpError,
};
// ===== FILE: cloudfunctions/huahua-api/lib/save.js =====
/**
 * 存档 pull / push 处理
 *
 * 数据模型（MongoDB collection: playerData）：
 *   {
 *     _id,
 *     userId: 'wx:...' | 'dy:...' | 'tap:...' | 'anon:...',
 *     schemaVersion: number,
 *     updatedAt: number (ms),
 *     baseRemoteUpdatedAt: number (ms), // 客户端本地修改基于的云端版本
 *     clientFingerprint: string,
 *     payload: { [key: string]: string }, // 客户端上行的白名单 key -> 原始 JSON 字符串
 *     payloadKeys: string[],
 *     lastWriteAt: number (ms, 服务端写入时间，用于排障)
 *   }
 */
const { httpError } = require('./http');
const { requireUser } = require('./auth');
const { getCollection } = require('./db');
const { getMaxBytes } = require('./config');
```

```
58324 async function handlePull(req) {
58325   const { userId, platform } = requireUser(req);
58326   const col = getCollection();
58327   const res = await col.where({ userId }).limit(1).get();
58328   const doc = (res && Array.isArray(res.data) && res.data[0]) || null;
58329   if (!doc) {
58330     return {
58331       userId,
58332       platform,
58333       exists: false,
58334       schemaVersion: 0,
58335       updatedAt: 0,
58336       payload: {},
58337       payloadKeys: [],
58338     };
58339   }
58340   return {
58341     userId,
58342     platform,
58343     exists: true,
58344     schemaVersion: doc.schemaVersion || 0,
58345     updatedAt: doc.updatedAt || 0,
58346     payload: doc.payload || {},
58347     payloadKeys: Array.isArray(doc.payloadKeys)
58348       ? doc.payloadKeys
58349       : Object.keys(doc.payload || {}),
58350     clientFingerprint: doc.clientFingerprint || '',
58351   };
58352 }
58353 async function handlePush(req) {
58354   const { userId, platform } = requireUser(req);
58355   const body = req.body || {};
58356   const schemaVersion = Number(body.schemaVersion);
58357   const updatedAt = Number(body.updatedAt);
58358   const baseRemoteUpdatedAt = Number(body.baseRemoteUpdatedAt || 0);
58359   const payload = body.payload;
58360   const clientFingerprint = String(body.clientFingerprint || '').slice(0, 200);
58361   const force = body.force === true;
58362   if (!Number.isFinite(schemaVersion) || schemaVersion <= 0) {
58363     throw httpError(400, 'BAD_SCHEMA', 'schemaVersion 非法');
58364   }
58365   if (!Number.isFinite(updatedAt) || updatedAt <= 0) {
58366     throw httpError(400, 'BAD_UPDATED_AT', 'updatedAt 非法');
58367   }
58368   if (!Number.isFinite(baseRemoteUpdatedAt) || baseRemoteUpdatedAt < 0) {
58369     throw httpError(400, 'BAD_BASE_REMOTE_UPDATED_AT', 'baseRemoteUpdatedAt 非法');
58370   }
58371   if (!payload || typeof payload !== 'object' || Array.isArray(payload)) {
58372     throw httpError(400, 'BAD_PAYLOAD', 'payload 必须是 object');
58373   }
```

```
// 体积上限（粗略按 JSON 序列化后字节数）
const size = Buffer.byteLength(JSON.stringify(payload), 'utf8');
const maxBytes = getMaxBytes();
if (size > maxBytes) {
  throw httpError(413, 'PAYLOAD_TOO_LARGE', `payload 超限: ${size}B > ${maxBytes}B`);
}
// 每个 value 必须是字符串（与客户端 PersistService 约定一致）
const payloadKeys = [];
for (const k of Object.keys(payload)) {
  const v = payload[k];
  if (typeof v !== 'string') {
    throw httpError(400, 'BAD_PAYLOAD_VALUE', `payload[${k}] 必须是字符串`);
  }
  payloadKeys.push(k);
}
const col = getCollection();
const existingRes = await col.where({ userId }).limit(1).get();
const existing = (existingRes && Array.isArray(existingRes.data) && existingRes.data[0]) || null;
if (existing && !force) {
  const prevUpdatedAt = Number(existing.updatedAt) || 0;
  if (updatedAt < prevUpdatedAt || baseRemoteUpdatedAt < prevUpdatedAt) {
    // 服务端已有更新的版本 —— 拒绝，并把远端版本回传让客户端合并
    throw Object.assign(
      httpError(
        409,
        'STALE_UPDATE',
        `服务端已有更新版本 remote=${prevUpdatedAt} > local=${updatedAt}, base=${baseRemoteUpdatedAt}`,
      ),
      {
        data: {
          remote: {
            schemaVersion: existing.schemaVersion || 0,
            updatedAt: prevUpdatedAt,
            payload: existing.payload || {},
            payloadKeys: Array.isArray(existing.payloadKeys)
              ? existing.payloadKeys
              : Object.keys(existing.payload || {}),
          },
        },
      },
    );
  }
}
const now = Date.now();
const docData = {
  userId,
  platform,
  schemaVersion,
  updatedAt,
  baseRemoteUpdatedAt,
```

```
    clientFingerprint,
    payload,
    payloadKeys,
    lastWriteAt: now,
  };
  if (existing && existing._id) {
    await col.doc(existing._id).update(docData);
    return { userId, updatedAt, savedAt: now, mode: 'update', sizeBytes: size };
  }
  const addRes = await col.add(docData);
  return {
    userId,
    updatedAt,
    savedAt: now,
    mode: 'insert',
    sizeBytes: size,
    _id: addRes && (addRes.id || (addRes._id)),
  };
}
module.exports = {
  handlePull,
  handlePush,
};
// ===== FILE: cloudfunctions/huahua-api/scripts/fake-tcb.js =====
/**
 * @cloudbase/node-sdk 的内存假实现，仅覆盖本项目用到的接口。
 */
const SYMBOL_CURRENT_ENV = Symbol('SYMBOL_CURRENT_ENV');
class FakeCollection {
  constructor(name, store) {
    this.name = name;
    this.store = store;
    this._filter = null;
    this._limit = 0;
  }
  _clone() {
    const c = new FakeCollection(this.name, this.store);
    c._filter = this._filter;
    c._limit = this._limit;
    return c;
  }
  where(cond) {
    const c = this._clone();
    c._filter = cond;
    return c;
  }
  limit(n) {
    const c = this._clone();
    c._limit = n;
    return c;
```

```
  }
  async get() {
    const all = this.store.docs.filter((d) => matches(d, this._filter));
    const out = this._limit > 0 ? all.slice(0, this._limit) : all;
    return { data: out.map((d) => ({ ...d })) };
  }
  doc(id) {
    return {
      update: async (data) => {
        const idx = this.store.docs.findIndex((d) => d._id === id);
        if (idx === -1) return { updated: 0 };
        this.store.docs[idx] = { ...this.store.docs[idx], ...data };
        return { updated: 1 };
      },
      remove: async () => {
        const before = this.store.docs.length;
        this.store.docs = this.store.docs.filter((d) => d._id !== id);
        return { deleted: before - this.store.docs.length };
      },
      get: async () => {
        const d = this.store.docs.find((x) => x._id === id);
        return { data: d ? [{ ...d }] : [] };
      },
    };
  }
  async add(data) {
    const _id = 'mock_' + Math.random().toString(36).slice(2, 10);
    this.store.docs.push({ _id, ...data });
    return { id: _id };
  }
}
function matches(doc, filter) {
  if (!filter) return true;
  for (const k of Object.keys(filter)) {
    if (doc[k] !== filter[k]) return false;
  }
  return true;
}
function init(_opts) {
  const stores = {};
  return {
    database() {
      return {
        collection(name) {
          if (!stores[name]) stores[name] = { docs: [] };
          return new FakeCollection(name, stores[name]);
        },
      };
    },
  };
```

```
58524 }
58525 module.exports = { init, SYMBOL_CURRENT_ENV };
58526 // ===== FILE: cloudfunctions/huahua-api/scripts/mock.js =====
58527 /**
58528 * 本地联调 mock：不依赖 CloudBase，把 save 集合读写替换为内存。
58529 * 运行：node scripts/mock.js
58530 *
58531 * 依赖：node 18+（内置 fetch）；不需要真实的 WX/TT AppSecret（会走 anon 登录路径）。
58532 */
58533 process.env.HUAHUA_JWT_SECRET = process.env.HUAHUA_JWT_SECRET || 'dev-secret-do-not-use-in-prod';
58534 process.env.HUAHUA_TOKEN_TTL_SEC = process.env.HUAHUA_TOKEN_TTL_SEC || '3600';
58535 // 劫持 @cloudbase/node-sdk，伪造内存集合
58536 const Module = require('module');
58537 const origResolve = Module._resolveFilename;
58538 Module._resolveFilename = function (request, parent, ...rest) {
58539  if (request === '@cloudbase/node-sdk') {
58540   return require.resolve('./fake-tcb.js');
58541  }
58542  return origResolve.call(this, request, parent, ...rest);
58543 };
58544 const { main } = require('../index');
58545 async function call(path, body, headers = {}) {
58546  const event = {
58547   httpMethod: 'POST',
58548   path,
58549   headers: { 'content-type': 'application/json', ...headers },
58550   body: JSON.stringify(body || {}),
58551   isBase64Encoded: false,
58552  };
58553  const res = await main(event, {});
58554  const parsed = res && res.body ? (() => { try { return JSON.parse(res.body); } catch { return r...
58555  console.log(`[${path}]`, res && res.statusCode, JSON.stringify(parsed));
58556  return { statusCode: res && res.statusCode, body: parsed };
58557 }
58558 (async () => {
58559  // 1) anon 登录
58560  const login = await call('/login', { platform: 'anon', anonId: 'testuser-0001' });
58561  const token = login.body && login.body.data && login.body.data.token;
58562  if (!token) throw new Error('login failed');
58563  const auth = { authorization: `Bearer ${token}` };
58564  // 2) pull（期望 exists: false）
58565  await call('/save/pull', {}, auth);
58566  // 3) push v1
58567  const now = Date.now();
58568  await call('/save/push', {
58569   schemaVersion: 1,
58570   updatedAt: now,
58571   baseRemoteUpdatedAt: 0,
58572   clientFingerprint: 'mock|dev',
58573   payload: { huahua_save: JSON.stringify({ ver: 1, coin: 100 }) },
```

```
58574   }, auth);
58575   // 4) pull 再次（期望 exists: true，数据一致）
58576   await call('/save/pull', {}, auth);
58577   // 5) push 旧 updatedAt（期望 409 STALE_UPDATE）
58578   await call('/save/push', {
58579     schemaVersion: 1,
58580     updatedAt: now - 1000,
58581     baseRemoteUpdatedAt: now,
58582     clientFingerprint: 'mock|dev',
58583     payload: { huahua_save: JSON.stringify({ ver: 1, coin: 50 }) },
58584   }, auth);
58585   // 6) push 新 updatedAt 但旧云端基线（期望 409 STALE_UPDATE，防本地旧缓存回写）
58586   await call('/save/push', {
58587     schemaVersion: 1,
58588     updatedAt: now + 1000,
58589     baseRemoteUpdatedAt: 0,
58590     clientFingerprint: 'mock|dev',
58591     payload: { huahua_save: JSON.stringify({ ver: 1, coin: 20 }) },
58592   }, auth);
58593   // 7) push 新 updatedAt 且基于当前云端版本（期望 ok）
58594   await call('/save/push', {
58595     schemaVersion: 1,
58596     updatedAt: now + 1000,
58597     baseRemoteUpdatedAt: now,
58598     clientFingerprint: 'mock|dev',
58599     payload: {
58600       huahua_save: JSON.stringify({ ver: 1, coin: 200 }),
58601       huahua_checkin: JSON.stringify({ day: 3 }),
58602     },
58603   }, auth);
58604   // 8) pull final
58605   await call('/save/pull', {}, auth);
58606   // 9) 无 token → 401
58607   await call('/save/pull', {});
58608   // 10) 超大 payload → 413
58609   const big = 'x'.repeat(300 * 1024);
58610   await call('/save/push', {
58611     schemaVersion: 1,
58612     updatedAt: now + 2000,
58613     baseRemoteUpdatedAt: now + 1000,
58614     clientFingerprint: 'mock|dev',
58615     payload: { huahua_save: big },
58616   }, auth);
58617   // 11) 健康检查
58618   await call('/health', {});
58619   console.log('\n[mock] 全部用例运行完毕');
58620 })().catch((e) => {
58621   console.error('[mock] 失败:', e);
58622   process.exit(1);
58623 });
```